

Project Number: 10372

Real-Time Sensor Based Spinning-Top Platform

Embedded Software, Telemetry, and System Optimization

Sensors ● FreeRTOS ● WebSocket Telemetry ● Android Visualization ● POV Display

Written by Keselman Liya & Jerushalmi Iddo

Spring 25/26

Supervisor: Mizrachi Boaz



Table of Contents

Real-Time Sensor-Based Spinning-Top Platform.....	0
1. Abstract.....	3
2. Motivation	3
3. System Goals and the Project's Vision.....	4
4. General Overview.....	5
4.1. Hardware Block Diagram (overview).....	5
4.2. Software Block Diagram (overview)	6
5. Components.....	7
5.1. ESP32 Dev Module.....	7
5.2. Charge Controller 3S	7
5.3. Buck Boost.....	7
5.4. Motor Driver L9110S.....	8
5.5. Lithium Ion Battery 18650.....	9
5.6. VL53L0XV2 Sensor	9
5.7. TF LUNA LiDAR Sensor	10
5.8. MPU6050 Sensor	10
5.9. Hammers	10
5.10. Leds.....	11
6. Protocols.....	12
6.1. Local Communication Protocols.....	12
6.2. Web Communication Architecture.....	14
7. Mechanical Overview.....	16
7.1. Placing and Sizing.....	16
7.2. 3D Modelling and Structural Modifications	16
7.3. Optional Positions.....	18
7.4. Physical Balancing Device	19
8. Electrical Overview	20
8.1. Schematic	20
8.2. Layout	26
9. Software Overview	28
9.1 Android Mobile Application.....	28
9.1.1 Application Architecture and Workflow	28
9.1.2 Dual-Network Communication and Telemetry Pipeline	29
9.1.3 Android Visualization and Connection Optimization	31
9.1.4 Application Interface and Operational Guide.....	31
9.2 Embedded Firmware Component (ESP32)	32
9.2.1 FreeRTOS Task Architecture and Core Assignment	32
9.2.2 Shared-State Synchronization and Snapshot Distribution.....	33
9.2.3 Data Logging Optimization and Memory Buffering.....	34
9.2.4 Sensor Fusion and Attempt for Drift Correction via Spatial Vector Integration.....	35

9.2.5 Binary Telemetry Protocol and Bandwidth Efficiency	36
9.2.6 Telemetry Throttling and UI Synchronization:	36
9.2.7 Sensor Porting and I2C Compatibility:	37
9.2.8 POV Display Mechanism	37
10. Testing and Performance	38
10.1. Independent Testing	38
10.2. Integration Test	41
11. Results, Conclusions and Optional Solutions	45
11.1 Power System & Reliability.....	45
11.2 Mechanical Balancing & Material Selection	45
11.3 Actuation and Hardware Evolution.....	45
11.4 Firmware and I/O Constraints.....	46
11.5 Hardware Fixes	46
11.6 Frequency Constraints and Sampling-Rate Tradeoffs	46
11.7 Real-Time Software Pipeline Optimization.....	47
11.8 Component Reliability and Procurement Strategy	49
12. Thanks and Recognition	52
13. Bibliography	53
13.1. Sources to 3D files	53
13.2. Datasheets	53
13.3. ESP32 Safe Ports	53
14. Appendix.....	54

1. Abstract

[Presentation Link](#)

This project presents the design, optimization, and implementation of an advanced autonomous spinning top system, built as a direct continuation of the initial prototype developed in our previous project. The primary objective was to transform a basic hardware setup into a high-performance, power-stable, and precisely balanced robotic platform capable of dynamic active stabilization.

On the hardware front, the energy subsystem was completely redesigned, transitioning from a 1-cell (1S) Lithium-Ion configuration to a high-voltage 3-cell (3S) series architecture governed by a 3S battery management system (BMS). This upgrade elevated the nominal system voltage to 12[V], significantly expanding the available power envelope and increasing the electromagnetic torque of the stabilization actuators ("hammers") for aggressive angular corrections. To support this architecture without a full PCB redesign, the legacy board from Project A was adapted with targeted hardware modifications, including protection resistors for the microcontroller. Mechanically, the structure was optimized for an unobstructed central-shaft alignment, replacing provisional balancing methods with an iterative static balancing procedure using polymer clay to achieve "random equilibrium" and minimizing rotational vibrations.

From a software perspective, a real-time concurrent architecture was developed for the ESP32 microcontroller to handle multi-task synchronization efficiently. Advanced synchronization algorithms were implemented for real-time LED control to achieve a stable, drift-free Persistence of Vision (POV) display. Concurrently, the system managed sensor-data acquisition, wireless streaming to an Android interface, and local CSV data logging while protecting shared variables from race conditions.

Experimental results demonstrated a significant increase in electromagnetic correction force and achieved spin time of around 30 seconds which maintained high angular stability. Finally, the project outlines critical engineering insights regarding ESP32 GPIO strapping pin constraints, the necessity of an I/O expander for future scalability, and procurement strategies for high-load power management components in enclosed robotic systems.

2. Motivation

This project represents the second phase of development, serving as a direct continuation of the work established in Project A. During the initial phase, the foundations were laid through the assembly of the first hardware prototype and the validation of core algorithms for the spinning top system. In this current stage, we are significantly enhancing the system's performance by tripling the spinning top's power output and performing the necessary hardware upgrades to support this increased capacity. Furthermore, we aim to refine the mechanical design, ensure full application integration, and implement a drift-free Arduino POV (Persistence of Vision) display. This effort is part of a larger collaborative framework involving multiple student groups, with the technical documentation and source code from the preliminary phase documented in the **Project A [GitHub repository](#)**.

3. System Goals and the Project's Vision

Project Vision:

The vision behind this project is to transform a traditional mechanical object into a sophisticated, autonomous robotic system. By integrating advanced sensing, high-torque actuation, and real-time processing, the project aims to create a spinning top that can maintain its balance indefinitely through active stabilization. Project B represents a major leap toward this goal, moving from a low-voltage prototype to a high-performance, power-stable system capable of handling significant physical dynamics.

Key System Goals:

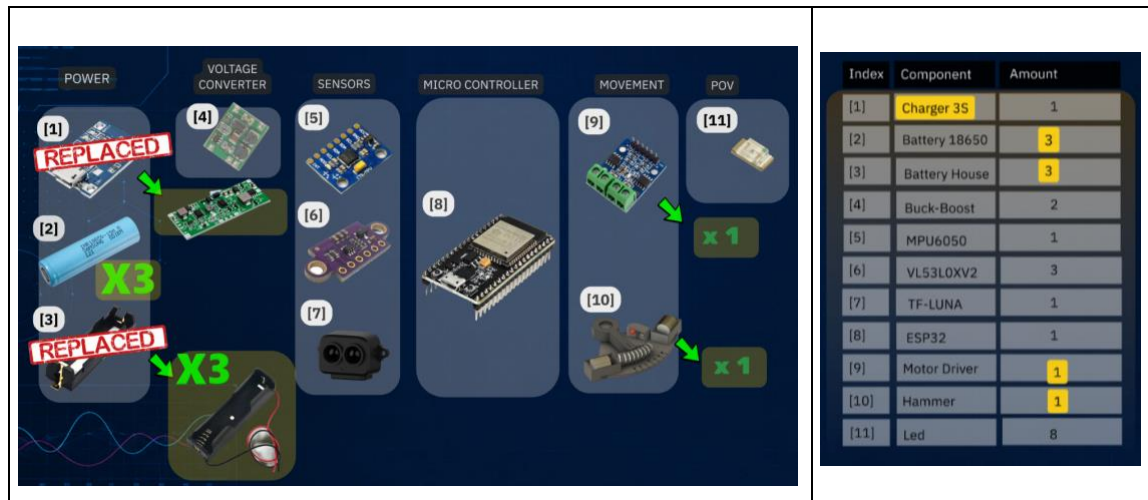
To realize this vision, the following primary goals were defined for the current development phase:

- **Active Stabilization:** Achieving a high degree of vertical stability by utilizing real-time feedback from motion sensors to trigger precise electromagnetic corrections.
- **Power Architecture Upgrade (The 3S Shift):** Transitioning the energy subsystem from a 1-cell (1S) Lithium-Ion configuration to a 3-cell (3S) series configuration (12[V]) governed by a 3S charge controller. This expands the available power envelopes to provide the high current and torque that will be required in the future for aggressive angular correction and long-term operational stability without requiring a full PCB redesign.
- **Mechanical Redesign & Rotational Stability:** Redesigning the mechanical structure to enable a continuous, unobstructed central-shaft alignment. This includes optimizing the internal battery layout and replacing provisional balancing methods with a permanent, mechanically robust static balancing solution.
- **Advanced Software Synchronization & Concurrency:** Developing a real-time concurrent architecture optimized for maximum execution speed, low latency, and minimum lagging. The system must seamlessly synchronize critical, non-blocking tasks including real-time sensor data acquisition, LED POV control, data logging, and wireless transmission.
- **High-Fidelity Telemetry & Dual-Stream Data Synchronization:** Implementing a multi-tier data logging and visualization mechanism with strict cross-stream consistency:
 - **High-Density Local Logging:** Capturing comprehensive high-resolution snapshots of sensor measurements and calculated engineering values every 20 milliseconds, written directly to a local CSV file for accurate offline analysis, debugging, and performance evaluation.
 - **Throttled Real-Time Streaming:** Transmitting a synchronized, downsampled data stream via WebSocket to a dedicated Android application interface. This balances network reliability and visual clarity, ensuring fluid real-time graph visualization without blocking or stalling the microcontroller's core control loops.
 - **Drift-Free Visual Display:** Implementing advanced synchronization algorithms for LED control using real-time rotational data to achieve a stable, drift-free Persistence of Vision (POV) display during high-speed rotation.

4. General Overview

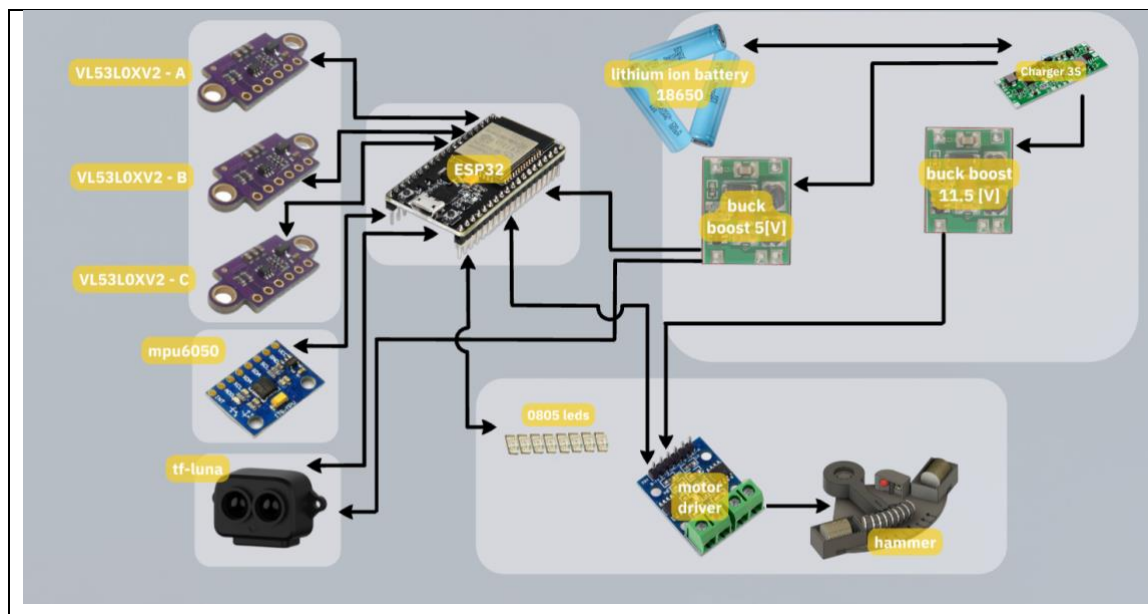
The main components are shown in the figure attached:

The system's main components are shown in the figure below, highlighting the hardware evolution from Project A to Project B, which primarily involves tripling the power output and upgrading the corresponding components to support the enhanced performance:

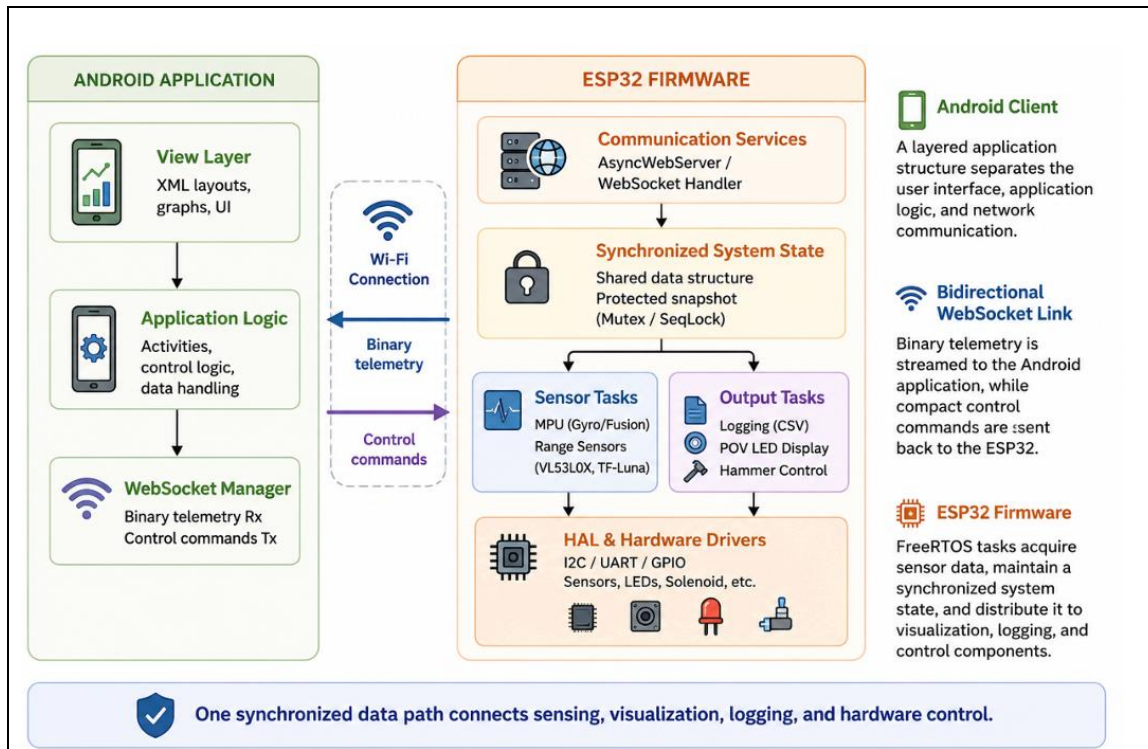


Before we explain the role of each component detailly (on next chapter), we will show a general view of the hardware and software blocks flow.

4.1. Hardware Block Diagram (overview)



4.2. Software Block Diagram (overview)



The system architecture is based on a modular, multi-layered framework, designed to support high-frequency sensor data processing while maintaining interface responsiveness.

At the firmware level, the system implements an asymmetric task distribution model utilizing the FreeRTOS environment.

This approach enables the isolation of time-critical operations, such as angular fusion and POV display rendering, from background tasks like communication management and data logging.

System dynamics are supported by a robust data pipeline, which employs protected shared memory to ensure temporal consistency and drift-free angular tracking.

Concurrently, the Android application utilizes a reactive architectural pattern, facilitating an asynchronous, full-duplex communication channel.

This design ensures that the high-throughput telemetry stream from the spinning top is efficiently processed and visualized, effectively balancing high-precision sampling with the constraints of the mobile interface.

The following sections provide a detailed examination of this architecture, starting with the firmware synchronization primitives (Section 9.1.2), data pipeline optimizations (Section 9.1.3), and concluding with the application's communication logic (Section 9.1.4).

5. Components

5.1. ESP32 Dev Module

The ESP32 is the central component of our device, selected for its numerous features, including multiple GPIO pins, built-in Wi-Fi modules, low cost, and the ability to generate PWM (Pulse Width Modulation) signals for motor control. Additionally, it supports I2C and UART communication protocols, which are essential for interfacing with various sensors and peripherals. We programmed the ESP32 to test the basic functionality of each component both individually and as part of the integrated system, ensuring the correctness of our device's routing and overall design.

5.2. Charge Controller 3S

A key hardware upgrade in Project B is the replacement of the previously used TP4056 charge controller. While the TP4056 was limited to managing only a single lithium cell (1S), the new 3S UPS module is designed to manage three batteries in series. This configuration effectively triples the system's voltage and power capabilities, supporting a maximum output of 36W.

Beyond basic charging, this module provides an essential Cell Balancing feature. By ensuring that all three cells maintain an identical voltage level, the controller prevents individual cell degradation and significantly extends the overall lifespan of the battery pack.

Furthermore, the module's integrated Uninterruptible Power Supply (UPS) capability ensures that the system receives a consistent 12[V] supply. This is particularly critical for the ESP32 and the POV display, as it prevents voltage fluctuations during high-load motor operations. This stability is vital for maintaining a drift-free visual output and preventing system resets when transitioning from external power to battery power.

5.3. Buck Boost

A DC-to-DC Buck-Boost converter can produce an output voltage magnitude that is either greater than or less than the input voltage magnitude. The specific converters integrated into our system support an input voltage range of 3[V] to 15[V], with an adjustable output range of 1[V] to 15[V]. On our PCB, we utilized two separate converters to isolate the power rails and ensure system stability:

Logic Power Rail (5[V]): This converter provides a steady 5[V] output to power the ESP32 microcontroller and the TF-Luna distance sensor. Separating this rail ensures that the logic components receive a stable voltage supply, independent of the fluctuations caused by the motors underload.

Motor Power Rail (V_{motor}): The second converter supplies power to the two motor drivers. In Project A, the system was powered by a single battery, and the optimal voltage was set to approximately 9[V]. In this current phase, by utilizing a 3-battery configuration (3S), we have upgraded the system to provide an output of slightly less than 12[V].

For a detailed technical explanation regarding the voltage selection and its impact on the system, please refer to Section 5.4: L9110S Motor Driver.

5.4. Motor Driver L9110S

The L9110S is a dual-channel motor driver module designed for controlling small-scale robotic systems and inductive loads. Each module features two independent driver chips, each capable of delivering up to 800mA of continuous current. The driver operates within a voltage range of 2. to 12[V] and is fully compatible with both 3.3[V] and 5[V] microcontroller logic.

In this project, we integrated two L9110S modules (totaling four output channels) to control the system's "hammers." These drivers receive PWM signals directly from the ESP32, allowing for precise control over the power delivered to the hammers. The mechanism functions by creating a high-voltage differential across the hammers' two inductors, triggering the mechanical action.

As referenced in the power management section (Section 4.1), the motor power rail (V_{motor}) is configured to provide slightly less than 12[V]. This specific voltage level was chosen because 12[V] is the absolute maximum rating for the L9110S chip. By maintaining the voltage just below this threshold, we maximize the hammers' impact force while ensuring the long-term reliability of the hardware components.

Hardware Integration Note:

During the testing phase, we identified a that there are some GPIO pins that are connected to the Motor Drivers and are not safe to use (GPIO[1], GPIO[3]) since they are used for flashing and debugging, and some pins that required to be HIGH during boot (GPIO[0], GPIO[5]) and LOW during programming (GPIO[0]). Therefore, they cannot be reliably used for PWM motor control.

For further information about risky pins, please read the following [article](#).

5.5. Lithium Ion Battery 18650

The system is powered by three 18650 Lithium-Ion rechargeable cells configured in a 3S (3-series) arrangement. The 18650 cell is a standard in high-performance applications due to its high energy density and ability to deliver significant discharge currents, which are essential for the project's increased power requirements.

To maintain the compact dimensions of the PCB and ensure a balanced mechanical structure, the batteries were arranged in an equilateral triangle configuration at the bottom of the board. Due to these strict spatial constraints, standard battery holders could not be utilized. Instead, the cells were soldered directly to one another to form a rigid, integrated pack. While this design optimizes the board's footprint, it introduces a maintenance trade-off: the batteries cannot be easily replaced or removed without desoldering the connections.

Furthermore, this specific mechanical integration requires 18650 cells with a "Flat Top" geometry (as shown in following image). Unlike "Button Top" batteries, the flat-head design allows for the precise alignment and stable contact points necessary for direct soldering within the triangular layout. To ensure safe operation, these batteries are managed by the 3S UPS Charge Controller (Section 4.2), which provides over-charge protection, over-discharge protection, and cell balancing.

Flat Top	Button Top
	

5.6. VL53L0X V2 Sensor

The VL53L0X V2 is a high-precision laser-ranging sensor based on Time-of-Flight (ToF) technology. It measures distances using an infrared laser, providing high accuracy and fast response times across a range of up to 4 meters, depending on surface reflectivity and lighting conditions.

In this project, these sensors are utilized to provide spatial data for the mobile application. By integrating the distance measurements with the spinning top's current rotation angle, the system generates a real-time graph in the app. This graph visualizes the distance of detected objects as a function of the rotation angle, effectively creating a 360-degree spatial map of the system's surroundings.

As noted in project A book, the VL53L0X V2 sensors have a sampling rate approximately one-third that of the higher-end TF-Luna sensor. To ensure a "fair" comparison and maintain high-resolution data, we utilize three VL53L0X V2 sensors. These sensors are powered by 3.3[V] from the ESP32 and communicate via the I2C protocol. Since they share the same communication line, they utilize a time-multiplexing approach where each sensor operates for one-third of the cycle while the others remain inactive, ensuring seamless data transmission to the ESP32 without bus conflicts.

5.7. TF LUNA LiDAR Sensor

The TF-Luna is a compact, high-performance LiDAR sensor based on the Time-of-Flight (ToF) principle. It is designed for precise distance measurement and environmental sensing.

- **Technology and Performance:** The sensor utilizes an infrared laser to measure distances up to 8 meters with high accuracy. Compared to budget ToF sensors like the VL53L0X, the TF-Luna offers a significantly higher sampling rate, making it an ideal choice for the real-time distance tracking required by the spinning top's control loop.
- **Communication Interface:** While the sensor supports both I2C and UART, it is configured to operate via the UART protocol in this project.
- **Power Supply:** The sensor receives a stable 5[V] supply from the system's voltage converter.
- **System Constraint:** As noted in the hardware architecture, the TF-Luna shares a communication line with LED 8. Due to the limited GPIO pins on the ESP32, these two components cannot be used simultaneously, requiring the firmware to prioritize the LiDAR for spatial sensing during active navigation.

5.8. MPU6050 Sensor

The MPU6050 is a 6-axis motion tracking device that integrates a 3-axis gyroscope and a 3-axis accelerometer. Within the project's hardware architecture, it is positioned as the primary source for spatial orientation data.

- **Operating Voltage:** The MPU6050 module is powered by the system's 5[V] rail, supplied by the dedicated voltage converter.
- **Future Stabilization Logic:** In future iterations of the control firmware, the MPU6050 will allow the system to calculate precise tilt angles and angular velocity along the X, Y, and Z axes. This data will enable the stabilization algorithm to trigger the electromagnetic "hammers" with micro-second precision.
- **Planned Feedback Loop:** Once the software integration is complete, the sensor will provide real-time motion feedback. This will permit the system to counteract gravitational pull and maintain an upright position even during high-speed rotations.
- **Communication:** It is designed to communicate via the I2C protocol. In future development, this will facilitate a seamless data flow between the motion sensors and the ESP32's processing cores.

5.9. Hammers

The "Hammer Modules" are a unique and central mechanical component of our project, responsible for the system's locomotion. Each module consists of a mobile arm that rotates around a central pivot point, with permanent magnets strategically positioned at its distal end.

Complementing the arm are two stationary coil housings. Each housing contains copper windings wrapped around an iron core, forming an electromagnet. The operational principle is based on electromagnetic induction: when an electric current flows through the coils, a concentrated magnetic field is generated, which attracts or rejects the magnets on the arm with significant force.

As the arm is rapidly toggled between the coils, it produces repeated, high-energy impacts on the edges of the coil housings. These controlled kinetic impulses are transferred to the system's frame, allowing the dreidel to move across the surface in a precise and controlled manner. The intensity and timing of these impacts are governed by the motor drivers and the ESP32's PWM signals, as discussed in previous sections.

5.10. Leds

The system's visual feedback is provided by a vertical array of eight 0805 SMD LEDs. These Surface Mount Devices were chosen for their compact footprint (2.0 x 1.25 mm), which allows for high-density vertical integration on the system's frame while maintaining a low mechanical profile.

Circuit Design and Active-Low Logic:

As illustrated in the system's schematic, the LED array is configured in an Active Low (Current Sinking) architecture. In this setup, each LED's cathode (K) is connected to a dedicated ESP32 GPIO pin, while the anode is tied to the +3.3[V] rail through a 330 [Ω] current-limiting resistor. To activate an LED, the ESP32 must pull the corresponding pin to a logical '0' (GND). This design leverages the ESP32's stability at sinking current, while the 330 [Ω] resistors ensure a safe operating range (5-10[mA]) to protect the components from thermal damage.

Mathematical Logic and Display Calculations:

The array functions as a Persistence of Vision (POV) display. By pulsing the LEDs in synchronization with the rotation, the system serves as a rotation counter. To provide clear feedback within the constraints of a single-digit vertical display, the system employs two primary mathematical operations:

Counter Increment Logic (Integer Division):

- The total number of rotations, captured by the Hall Effect sensor, is divided by 10 to determine the current count. Because this is integer division, any remainder is discarded, and the display only updates once a full cycle of 10 rotations is completed.
- Formula: $Counter\ Value = Total\ Rotation / 10$

Unit Digit Extraction (Modulo Arithmetic):

- Since the display shows a single digit at a time, the system isolates the least significant digit (LSD) using the modulo 10 operator.
- Formula: $Displayed\ Digit = Counter\ Value \pmod{10}$

Example Scenarios:

- At 40 rotations: $40 / 10 = 4$. The system applies 4 mod 10, and the display shows 4.
- At 92 rotations (In-between update): The system calculates $92 / 10 = 9$ (Due to integer division, the result remains 9). The system applies 9 mod 10, and the display shows 9. The display will continue to show 9 until the 100th rotation is reached.
- At 110 rotations: $110 / 10 = 11$. The system applies 11 mod 10, and the display shows 1.
- At 300 rotations: $300 / 10 = 30$. The system applies 30 mod 10, and the display shows 0.

Hardware Conflict (LED #8 & TF-Luna):

A critical hardware constraint exists for LED #8, which is tied to the same physical pin as the TF-Luna sensor's TXD line (ESP_LUNA_LED8_TXD). Because the LED is triggered by a low logic signal, any serial data transmitted by the sensor (high-speed toggling between '0' and '1') causes the LED to flicker according to the data traffic. Consequently, LED #8 cannot be used for a stable POV display while the sensor is active - a necessary trade-off to accommodate both sensing and display within the available GPIO limits.

6. Protocols

6.1. Local Communication Protocols

I²C and UART protocols operate entirely within the embedded hardware domain and are responsible for reliable acquisition of raw sensor data. The ESP32 functions as the central coordinator, collecting measurements from both communication channels, performing processing and synchronization, and preparing the data for higher-level transmission.

First, The ESP32 reads the sensors locally via I²C and UART and computes the required state variables.

Then, it publishes the resulting data over WiFi: it appends the data to a CSV log stored in SPIFFS (served to a computer via HTTP on request) and streams live telemetry as JSON message to the mobile app via WebSocket.

The local layer architecture establishes a clear separation between hardware-level communication, embedded processing, and network-level interaction. Such separation enhances modularity, maintainability, and scalability, while ensuring deterministic sensor acquisition and low-latency external.

I²C

Inter-Integrated Circuit (I²C) is a synchronous serial communication protocol widely used in embedded systems for short-distance communication between integrated circuits located on the same printed circuit board.

In the Smart Spinning-Top system, I²C serves as the primary internal communication bus between the ESP32 microcontroller and multiple onboard sensors. Its suitability for this project stems from its minimal wiring requirements, its ability to support multiple devices on a shared bus, and its master-controlled clock synchronization, which ensures reliable and deterministic communication within compact embedded environments.

In the current system architecture, the ESP32 operates as the single I²C master, while three VL53L0X time-of-flight distance sensors and one MPU6050 inertial measurement unit act as slave devices.

The bus is implemented using the ESP32's default I²C pins, GPIO21 for SDA (Serial Data Line- transfers data between the master and slaves) and GPIO22 for SCL (Serial Clock Line- synchronized data transfer, controlled by the master), operating at a high frequency of 400 [kHz] through the Wire library.

Since both lines are open-drain, pull-up resistors are required to maintain a defined logic HIGH level when the bus is idle. In this design, the necessary pull-ups are integrated within the sensor sections in the hardware schematic.

The following table shows the two main lines of I²C bus:

	SCL (Serial Clock Line)	SDA (Serial Data Line)
Purpose	Synchronizes data transfer.	Transfers data between the master and slaves.
Direction	Controlled by the master.	Bidirectional (data flows in both directions).
Function	The master generates clock pulses, and all connected slaves follow this clock to send/receive data.	The master sends a command to a specific slave, and the slave responds with data.
ESP32 Default Pin	GPIO22 (can be reassigned).	GPIO21 (can be reassigned).

A central design challenge arises from the fact that multiple VL53L0X sensors share the same factory default I²C address since they share same I²C bus. If enabled simultaneously, they would respond concurrently to the same address, causing bus contention and communication failure. To prevent this, the system performs a controlled initialization sequence using the sensors' XSHUT pins. At startup, all distance sensors are held in a disabled state. The ESP32 then activates each sensor individually, assigns it a new unique I²C address, and proceeds to the next device. Once all sensors have been assigned distinct addresses, normal multi-device communication on the shared bus can begin without conflicts.

The master initiates communication and controls the clock signal, while the slaves respond to the master's requests. Each slave has a unique I²C address, allowing the master to communicate with multiple devices using a shared bus. Each I²C transaction follows a structured sequence. Communication begins with a start condition generated by the master (ESP32) followed by the transmission of the target slave address and a read/write bit. The addressed slave acknowledges receipt, after which data bytes are exchanged, each followed by an acknowledgment bit. When the transaction is complete, the master issues a stop condition, releasing the bus. The SCL line, controlled by the master, provides clock synchronization, while the SDA line carries the actual data. If necessary, slave devices may apply clock stretching by holding SCL low until internal processing is complete.

Within the overall system architecture, I²C is responsible solely for local sensor acquisition. The ESP32 periodically queries each sensor, retrieves measurement registers, and stores the results in internal data structures. These measurements are subsequently processed for angle estimation, synchronization logic, and control algorithms. Only after local acquisition and processing are complete are the data logged to SPIFFS or transmitted wirelessly via WiFi using HTTP or WebSocket. Thus, I²C forms the foundational layer of the system's data acquisition pipeline.

UART

Universal Asynchronous Receiver-Transmitter (UART) is an asynchronous serial communication protocol that enables full-duplex point-to-point communication between two devices, means data can be sent and received simultaneously between two devices.

Unlike I²C, which allows multiple devices to share a common bus, UART establishes a dedicated communication channel between a transmitter and a receiver (one-to-one communication), and simplifies implementation by eliminating the need for device addressing. In this project, UART is used to interface with the TF-Luna LiDAR sensor, which continuously streams distance measurements to the ESP32.

As UART follows One-to-One Master-Slave communication Model, where the master controls the communication process and reads incoming data (data is transferred in packets), and the slave continuously transmits distance measurement data to the master, the UART interface consists of two main signal lines: TX (transmit) and RX (Receive).

TX (Transmit Data Line) : responsible for sending data to the receiver.

RX (Receive Data Line) : responsible for receiving data from the transmitter.

The sender always writes data over its TX line, and the receiver always reads data over its RX line.

The ESP32 includes multiple hardware UART interfaces, and in this system UART2 is configured using GPIO17 as TX2 and GPIO16 as RX2. Communication operates at a baud rate of 115200 bits per second. The TF-Luna sensor is configured to operate in UART mode by setting its configuration pin to 3.3 V, enabling continuous serial output of measurement frames.

Unlike I²C, UART does not rely on a shared clock signal, means it supports asynchronous communication. Instead, synchronization is achieved through precise timing based on an agreed baud rate (bits per second). Each transmitted frame begins with a start bit, followed by data bits (typically eight), an optional parity bit for error detection, and one or two stop bits that indicate the end of the frame. Because communication is asynchronous, both the transmitter and receiver must be configured to the same baud rate to avoid timing mismatches and data corruption.

This makes UART a simple and efficient protocol for real-time data exchange, commonly used in sensors, microcontrollers, and serial interfaces.

In the implemented system, the TF-Luna primarily operates in continuous transmission mode. After the ESP32 initializes the UART interface with the correct baud rate, the sensor repeatedly sends structured measurement packets over its TX line.

The ESP32 receives incoming bytes via its RX buffer, verifies packet integrity using checksum validation, and extracts the distance measurements. Although communication is mainly unidirectional in practice, the ESP32 retains the capability to send configuration commands to the sensor when parameter adjustments, such as frame rate or output mode, are required.

UART reliability in this design is ensured through proper baud rate synchronization, frame structure validation, checksum verification included in the TF-Luna packet format, and careful buffer management to guarantee complete frame reception before processing. Compared to I²C, UART provides a higher-rate streaming mechanism and eliminates the need for addressing, but it requires dedicated wiring and supports only one-to-one communication.

From an architectural perspective, UART serves as a continuous high-speed measurement channel that complements the polled, master-controlled nature of I²C. While I²C sensors are queried by the ESP32 when needed, the TF-Luna continuously supplies data, which is then integrated into the system's processing and synchronization algorithms before being logged or transmitted wirelessly.

Comparison: UART vs. I²C:

Feature	UART (TF LUNA)	I2C (VL53L0XV2, MPU6050)
Bus Type	Point-to-Point	Multi-Device Bus
Clock Signal	Not Required	Required (SCL)
Number of Wires	2 (TX, RX)	2 (SDA, SCL)
Addressing	No Addressing Needed	Each Slave Needs a Unique Address
Data Transfer	Continuous Streaming	Master Requests Data
Speed	Adjustable Baud Rate (e.g., 115200 bps)	Typically Fixed 100 [kHz] / 400 [kHz]

6.2. Web Communication Architecture

How Data Collected from Sensors is Transferred via WiFi to the Mobile App and the Local Computer

In the Smart Spinning-Top system, sensor measurements are acquired locally by the ESP32 from multiple peripherals using short-range board-level communication protocols. The distance sensors (VL53L0X) and the inertial measurement unit (MPU6050) communicate with the ESP32 over the I2C bus, while the TF-Luna distance sensor communicates over UART. These wired interfaces are used only inside the device and are responsible for reliable and low-latency sensor sampling at the embedded level. Once the ESP32 reads the raw measurements, it processes them and formats them into higher-level “telemetry snapshots”, which may include both direct sensor readings and computed state variables such as angle estimates, spin count, synchronization parameters, and diagnostic/debug fields.

Wi-Fi is used as the wireless transport layer that enables external devices to communicate with the ESP32 without any external networking hardware.

In this project, Wi-Fi does not replace I2C or UART. Instead, it is used after local sampling and processing to deliver logged data to a computer and to provide real-time telemetry and control to a mobile application. The ESP32 operates as a local server that exposes web communication endpoints, while external devices (computer or mobile phone) act as clients that connect over Wi-Fi.

Dual-Mode WiFi Operation (STA + SoftAP)

To support multiple deployment scenarios, the ESP32 operates in dual Wi-Fi mode (SoftAP + Station). In Station (STA) mode, the ESP32 connects to an existing local Wi-Fi router using predefined SSID and password credentials, and it receives a local IP address within that router’s network via DHCP. Any computer connected to the same router can reach the ESP32 using this IP address, enabling convenient access from a lab computer, or a local workstation.

In parallel, the ESP32 also creates its own wireless network in Access Point (AP) mode (SoftAP). In this mode, the ESP32 broadcasts an SSID and allows a mobile phone to connect directly to the device without relying on an external router. This provides a self-contained communication path that is especially useful for demonstrations, field testing, and scenarios where a router is unavailable. In both STA and AP modes, the ESP32 remains the server, while connected devices act as clients. The difference between the modes is primarily how the network is formed and which IP address is used to reach the ESP32.

Two Complementary Communication Interfaces: HTTP and WebSocket

To serve different use cases, the system exposes two complementary communication interfaces on top of Wi-Fi: an HTTP interface for snapshot-based monitoring and logging, and a WebSocket interface for real-time bidirectional communication. Both interfaces are hosted on the same ESP32 server instance (AsyncWebServer on port 80), but they target different endpoints and follow different communication paradigms.

This design allows the system to support both robust logging/debugging workflows and low-latency interactive control.

HTTP Interfaces: Snapshot-Based Logging to a Local Computer

For computer-based monitoring and offline analysis, the ESP32 continuously logs sensor data into a CSV file stored locally in SPIFFS (the ESP32 internal flash file system). During runtime, new measurement lines are appended to the CSV file, so the file is continuously updated inside the ESP32.

Importantly, this file does not move to any external server; it remains stored on the ESP32 and is served only when a client explicitly requests it.

When a computer accesses the ESP32 via a browser or a script using the ESP32’s IP address (for example, `http://<ESP32_IP>/` or `http://<ESP32_IP>/<file>.bin` depending on the configured endpoints), it sends a standard HTTP GET request. The ESP32 responds by reading the current content of the file from SPIFFS and returning it as an HTTP response. Because HTTP follows a request-response model, each request returns only a snapshot of the file content at the moment the request was handled. If additional sensor measurements are written to the CSV file immediately after the response has been sent, the client will not automatically receive them. To observe newly appended lines, the client must issue another GET request, typically by refreshing the browser manually or by running a script that repeatedly fetches the file at a chosen rate.

This snapshot-based approach is useful for logging, debugging, and post-processing (for example, loading the CSV into analysis tools).

It is simple and robust, but it is not designed for low-latency streaming or interactive control.

WebSocket Interface: Real-Time Telemetry and Control to a Mobile App

For real-time interaction with the Android application, the ESP32 exposes a WebSocket endpoint at `/ws`. WebSocket is a full-duplex protocol that operates over a single TCP connection and enables low-latency bidirectional communication between a client and a server. In contrast to HTTP, which requires the client to repeatedly initiate requests, WebSocket maintains a persistent connection and allows the server to push data to the client whenever new information is available.

A WebSocket session begins with an HTTP-based upgrade handshake. The mobile application initiates the connection by sending an HTTP GET request to the `/ws` endpoint with headers requesting a protocol upgrade (e.g., `Upgrade: WebSocket`). If the ESP32 accepts the connection, it replies with 101 Switching Protocols, and the communication transitions from HTTP to the WebSocket protocol. After this upgrade, both sides exchange WebSocket frames over the same persistent TCP channel, without the overhead of repeated HTTP requests.

Once the connection is established, the ESP32 streams telemetry updates to connected WebSocket clients in JSON format. These messages include live sensor readings and computed state variables such as angles, spin count, synchronization parameters, and additional diagnostic/debug values that are useful for validating the control and display behavior. The same channel also supports commands sent from the mobile application to the ESP32. Control messages are received inside the WebSocket event callback, parsed as JSON, and executed immediately.

In this project, commands may include hammer control requests, mode switching, and activity selection. This real-time bidirectional channel is essential for achieving immediate feedback and interactive control while the spinning top is operating.

Although the system typically communicates with a single primary client (the Android application), the implementation supports multiple WebSocket clients simultaneously. When multiple clients are connected, the ESP32 can broadcast telemetry to all active connections.

7. Mechanical Overview

7.1. Placing and Sizing

The mechanical design of the spinning top was centered around a custom round PCB with a diameter of 162 mm. This size represents an engineering compromise between creating a compact spinning device and providing sufficient surface area for the integrated sensors, while allowing internal volume for the power management system.

Mass Distribution and Rotational Dynamics: To ensure stable and prolonged rotation, the placement of components followed a strict radial symmetry strategy. Our primary objective was to concentrate mass near the outer edges of the PCB to increase the moment of inertia, which provides greater angular momentum and stabilizes the top during operation.

Power System Integration and Structural Balance:

- **3S Battery & Controller Assembly:** The core mass of the system consists of three Lithium-Ion 18650 batteries. To achieve inherent mechanical equilibrium, these batteries were positioned horizontally in an equilateral triangle formation.
- **Internal "Floating" Configuration:** To optimize space and maintain the center of gravity, the 3S Charge Controller was mounted in a "floating" configuration within the internal cavity of the spinning top, integrated directly with the battery pack. This tactical design choice was made to accommodate the upgraded 3S power system within the existing hardware framework, thereby avoiding the significant time and cost associated with designing and manufacturing a new PCB.
- **Electrical Bypass Logic:** Instead of using the original on-board controller footprint, the 3S controller's output was manually wired to the PCB: the OUT+ terminal was connected via an electrical lead to Pin 1 of the original controller's location, and the OUT- terminal to Pin 4. This allowed for a streamlined power delivery system that utilizes the existing board infrastructure while benefiting from the superior 3S charging logic.

Noise Mitigation for Sensors: High-sensitivity components were strategically placed to avoid electromagnetic interference. Specifically, sensors such as the VL53L0X, MPU6050, and TF-Luna LiDAR were positioned as far as possible from "noisy" components like the inductive coils and the L9110S motor driver.

7.2. 3D Modelling and Structural Modifications



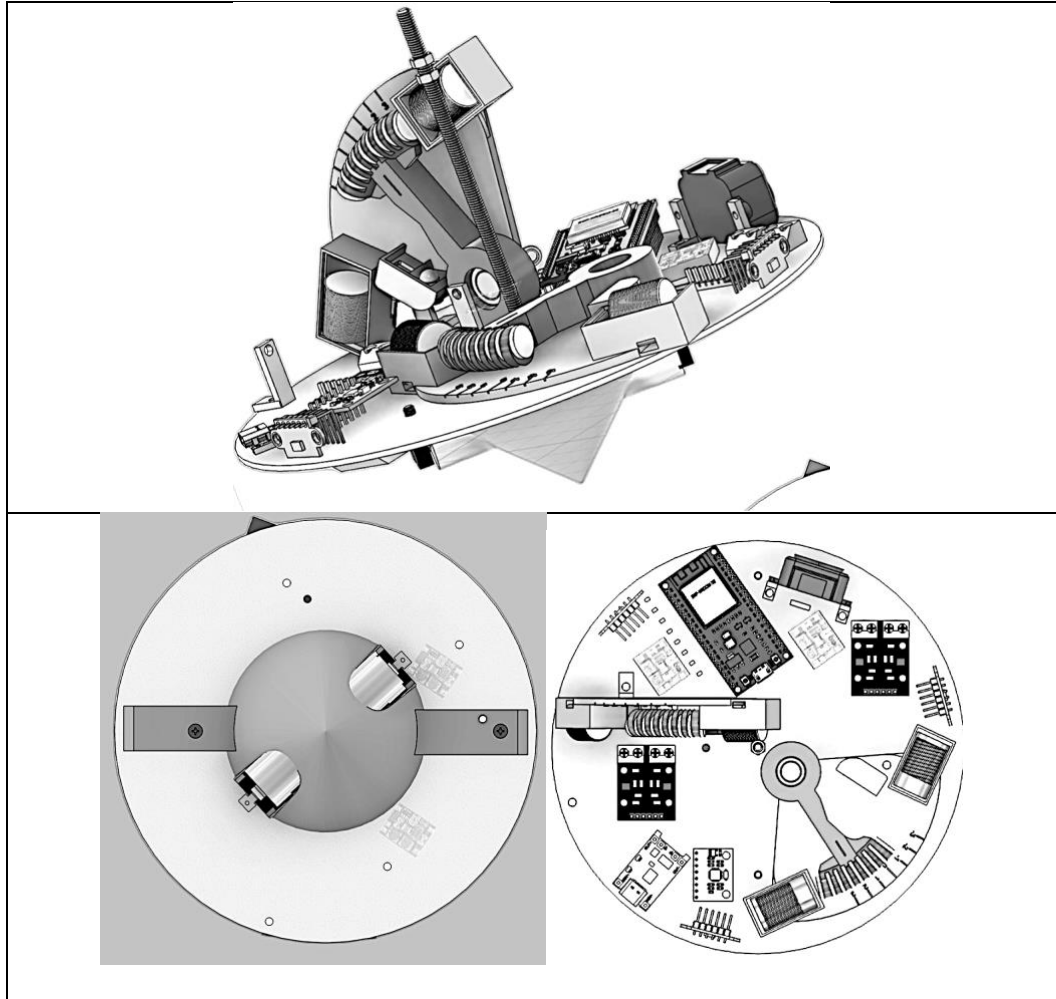
Evolution from Project A to Project B: In Project A, the 3D model was developed using Fusion 360, focusing on the PCB layout, handles, and cones. However, Project B required a complete structural overhaul to enhance durability and accommodate the upgraded power system (3S).

Mechanical Enhancements and Structural Challenges in Project B:

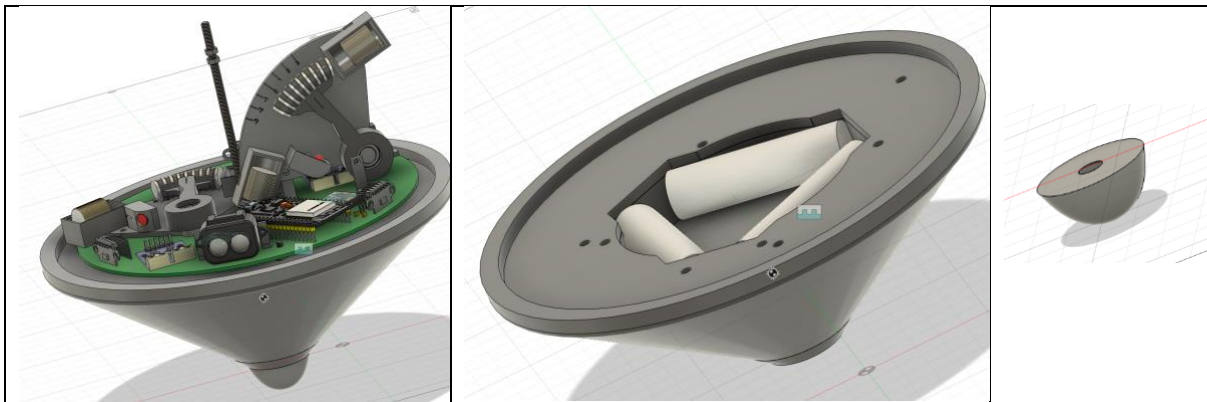
1. **Durable Outer Shell & Metallic Axis:** To improve structural integrity during high-speed rotation, we adopted a more robust outer casing and a professional metallic rotating leg (below the central screw) designed by another group.
2. **Internal Chamber Redesign:** The interior of the shell was modified with internal grooves carved into the "belly" of the spinning top to house the three 18650 batteries and the 3S Charge Controller.
3. **Mass Symmetry and Balancing Efforts:** To maintain radial balance, we attempted to arrange the three batteries in an equilateral triangle formation around the center. However, we encountered two main practical challenges:
 - **Fixation Difficulty:** It was challenging to firmly secure the batteries in this exact geometry, leading to minor shifts under centrifugal forces.
 - **Asymmetry caused by the 3S Module:** The integration of the 3S Charge Controller within the same cavity further disrupted the symmetry, as its additional mass created an uneven weight distribution relative to the battery triangle.

4. **Cable Management via Central Screw:** To deliver power to the PCB without fabricating a new board, we utilized the central screw that serves as the top's axis. The electrical leads for battery charging and power delivery were routed through the existing slot of this structural screw, ensuring a functional connection while maintaining the mechanical integrity of the axis.

The previous 3D model:



The current 3D model:

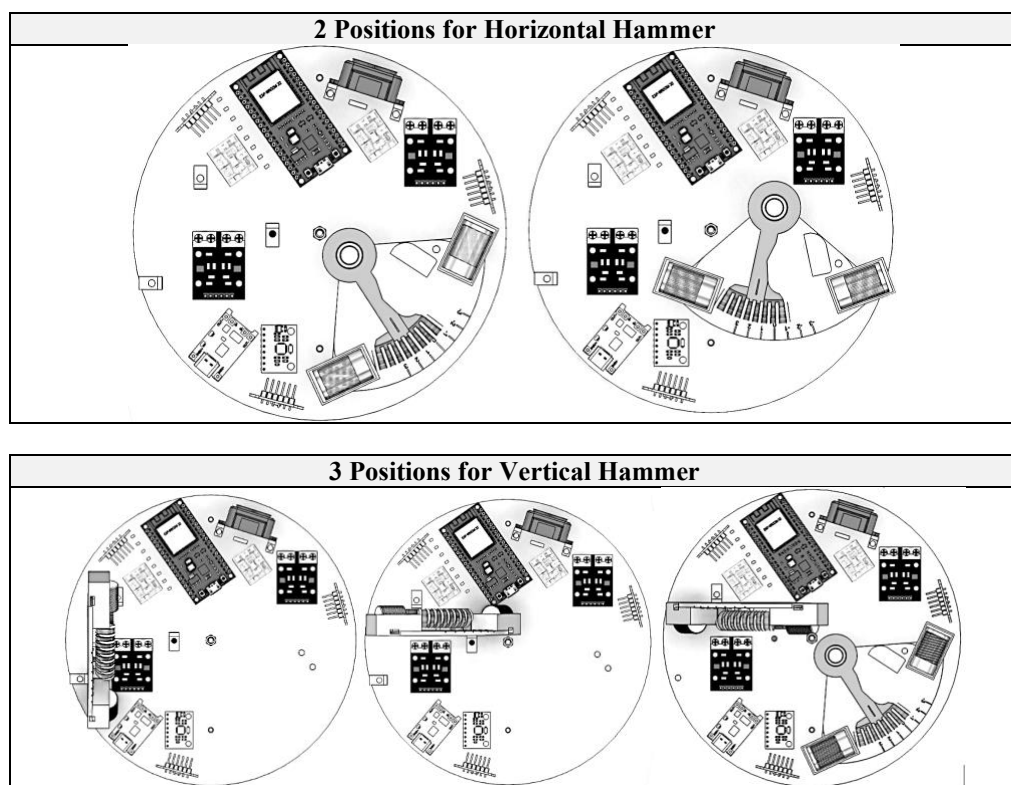


7.3. Optional Positions

Project A Design Intent: In the initial design phase, the system was engineered to support high physical adaptability, offering 2 positions for the horizontal hammer and 3 positions for the vertical hammer (totaling 6 potential configurations). This flexibility was intended to allow experimental optimization of the spinning-top's rotational stability.

Project B Technical Constraint (Pin Conflict): During the implementation of Project B, a critical hardware conflict was identified regarding the second motor driver (U2).

- **The Issue:** The control inputs for motor driver U2 were connected to GPIO 18, 5, 4, and 0.
- **The Consequence:** While the PCB physically supports the driver, these specific pins on the ESP32 presented significant functional issues. Specifically, GPIO 0 is a "strapping pin" required for the bootloader sequence; connecting it to a motor driver circuit often interferes with the chip's ability to boot or flash firmware. Additionally, the internal state of these pins during startup made them unsuitable for reliable PWM motor control in our specific PCB layout.
- **System Impact:** Consequently, the hammer assembly associated with U2 was rendered non-functional. The project proceeded by utilizing the primary operational motor driver (U1), concentrating the control logic and rotational stabilization on the active hammer system. This serves as a vital engineering lesson regarding the careful selection of GPIOs, particularly avoiding strapping and reserved pins for high-current peripheral drivers.



7.4. Physical Balancing Device

Despite our efforts to maintain radial symmetry during the design phase, the internal configuration of the three 18650 batteries and the "floating" 3S controller created an inherent mass imbalance. To ensure stable rotation and prevent wobbling (precession), we developed a dedicated Calibration Rig and implemented a manual balancing process.

The Calibration Rig: The rig consists of two vertical supports with precision-aligned slots. The central axis of the spinning top (the structural screw) is placed within these slots, allowing the device to rotate freely with minimal friction. This setup enables us to identify the "heavy" and "light" spots of the spinning top under the influence of gravity.

Manual Balancing Process (Polymer Clay Method): To achieve mass equilibrium, we used high-density polymer clay as a counterweight. The balance was conducted in two stages:

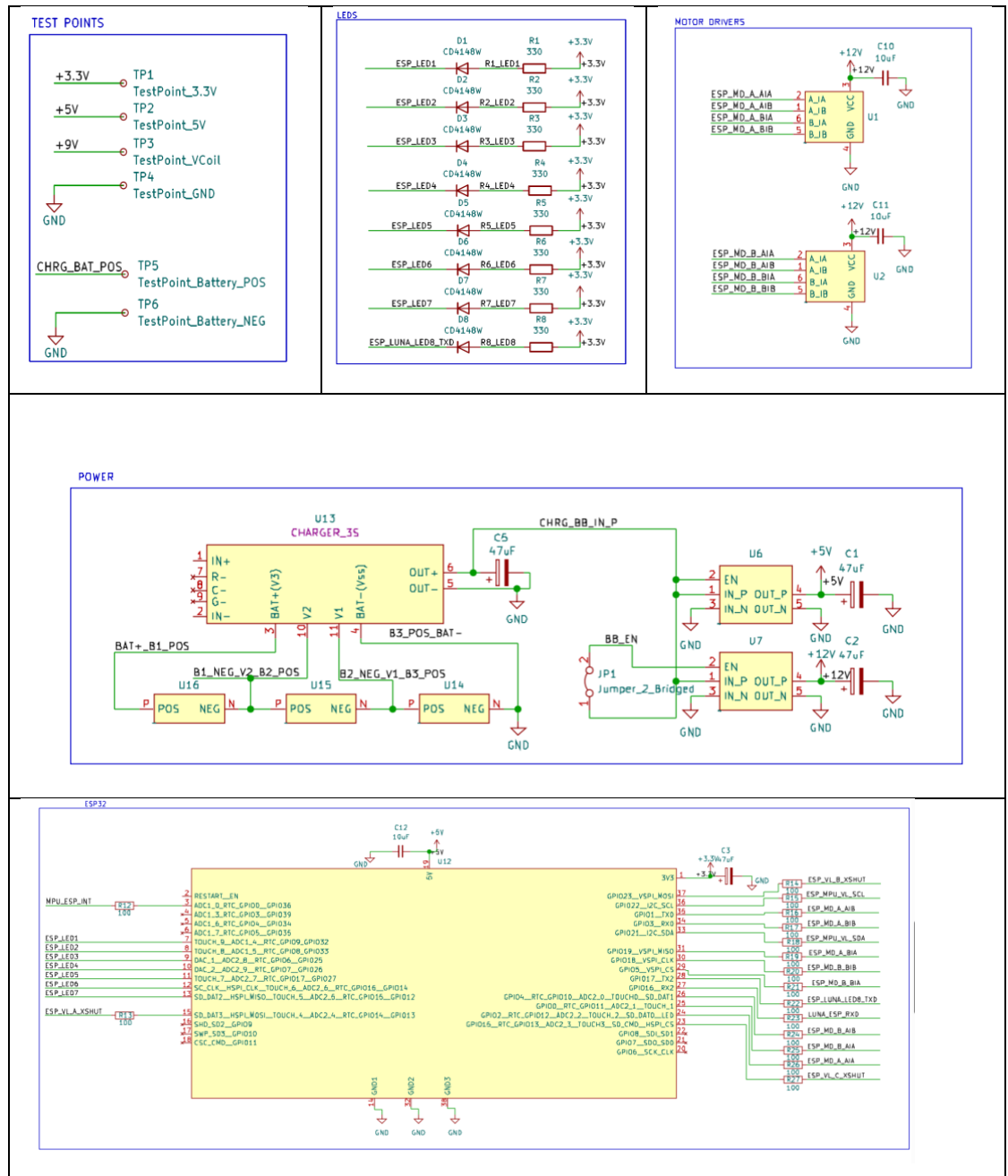
1. **Imbalance Identification:** The spinning top was given a controlled, gentle manual rotation on the rig. Due to gravity, the top would consistently come to rest with its heaviest point at the bottom and its lightest point at the top.
2. **Iterative Correction:** By repeating this process multiple times, we identified the specific angular sector that consistently ended up in the uppermost position (the lightest zone). We then applied measured amounts of polymer clay to this sector.
3. **Validation:** The process was repeated until the spinning top no longer stopped at a predictable angle, indicating that the mass was distributed uniformly around the axis of rotation.

This physical calibration was essential to compensate for the hardware-induced asymmetry, ensuring that the motor-driven stabilization system could operate on a fundamentally balanced platform.

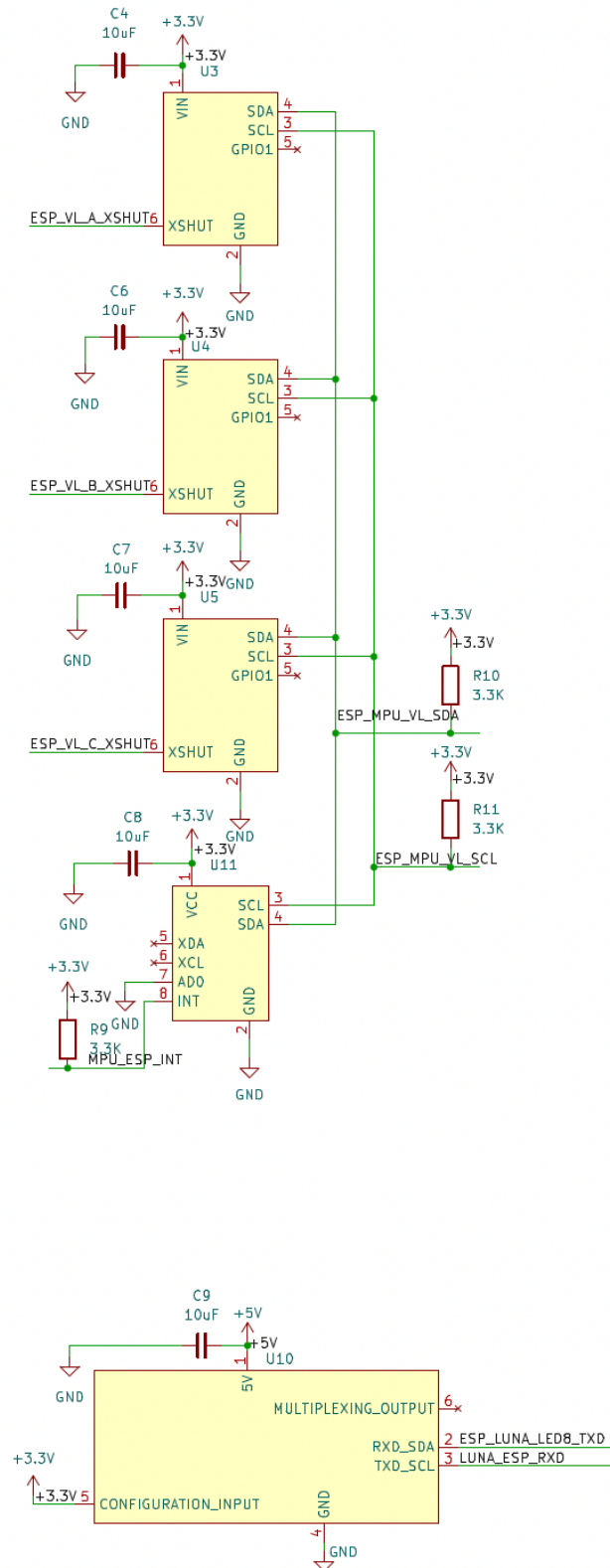
8. Electrical Overview

8.1. Schematic

The full schematic file is attached to [GitHub repository](#).



DISTANCE SENSORS



Power Block:

Power Management Unit (PMU):

The Power Management Unit is the backbone of the spinning top, responsible for converting raw battery voltage into stabilized rails for logic and high-power actuators. In Project B, this unit was significantly upgraded to a 3S configuration to support higher torque and longer operational time.

3S Battery Array & Protection (U13-U16):

The system is powered by a series of three 18650 Lithium-Ion cells (U14, U15, U16). To manage this high-capacity source, we integrated a 3S Battery Management System (BMS) module (U13).

- Balance Charging: Nodes V1 and V2 allow the controller to monitor and balance individual cell voltages, ensuring longevity and safety.
- The "Floating" Retrofit: Due to the transition from the legacy Project A power system, the 3S controller (U13) was implemented as a "floating" module within the internal shell. Its outputs, OUT+ (Pin 6) and OUT- (Pin 5), were manually wired to the PCB's main power entry points (formerly Pins 1 and 4 of the old controller footprint). This bypass allowed us to utilize the 11.1V-12.6V potential without redesigning the entire PCB.
- Input Stability (C5): To ensure a stable voltage source for the entire board and filter initial noise from the battery management output, a high-capacity C5 (47 μ F) capacitor is placed at the main power entry point.

Dual-Rail Regulation Strategy:

The power distribution is divided into two independent buck-boost regulation stages, along with a secondary power supply stage:

1. Higher-Power Rail (+12[V]) - U7:

- Role: Supplies power to the electromagnetic coils and the L9110S motor drivers.
- Power Rail Stability (C2): A C2 (47 μ F) capacitor is placed at the output of this regulator. It serves as an energy reservoir to maintain a stable 12[V] rail, which is critical during the high-current demands and noise generated by the actuators.
- The BB_EN Jumper (JP1): This jumper allows the user to physically enable or disable the high-power rail. During firmware development and debugging, the jumper can be removed to prevent accidental activation of high-power actuators, thereby protecting both the hardware and the developer. An equivalent effect can also be achieved by removing the U7 buck-boost converter

2. Lower-Power Rail (+5[V]) - U6:

- Primary Function: Provides a stabilized 5[V] supply to the ESP32 and the TF-Luna LiDAR.
- Logic Stability (C1): A high-capacity decoupling capacitor, C1 (47 μ F), is placed at the output to suppress voltage ripples caused by the high-current pulses of the coils. C1 and C2 work in tandem to ensure that while C2 stabilizes the power rail, C1 provides a clean, filtered supply for the sensitive logic components.

3. Secondary Power Supply (+3.3[V]): The MPU6050 and VL53L0X sensors receive a regulated 3.3[V] supply, derived from the ESP32's onboard voltage regulator. This ensures signal compatibility with the ESP32's I/O pins.

Distance Sensor Block:

The sensing block is the "eyes" of the system, comprising five sensors of three different types. This suite provides the real-time feedback loop necessary for balancing and navigation.

- I2C Bus Configuration (VL53L0X & MPU6050): The system integrates three VL53L0X Time-of-Flight (ToF) sensors (U3, U4, U5) and one MPU6050 IMU (U11). These sensors are powered by the 3.3[V] output of the ESP32. Due to the limited number of available GPIOs, all three VL sensors share the same I2C data line. Since they have identical default addresses, we implemented a multiplexing strategy using their XSHUT pins: the ESP32 initializes them sequentially, allowing each sensor to share the bus equally so that each transmits data for one-third of the cycle time.
- The Role of Pull-up Resistors (R9, R10, R11): As certain ESP32 inputs and outputs cannot inherently maintain a "pure" logical '1' (High) due to internal architecture and noise, we integrated 3.3k Ω pull-up resistors. These resistors serve two main purposes:
 1. Overcoming Open-Drain Architecture: Since I2C is an "Open-Drain" protocol, resistors R10 and R11 (on SDA/SCL lines) ensure the bus remains at 3.3[V] when not actively pulled low, preventing the lines from "floating."
 2. Signal Integrity: They provide a stronger pull than the ESP32's internal resistors, ensuring a stable logical '1' even amidst electromagnetic interference. Additionally, R9 stabilizes the MPU6050's interruption line to prevent false triggers.
- UART Sub-system (TF-Luna LiDAR): The fifth sensor is the TF-Luna LiDAR (U10), used for long-range measurement. Unlike the I2C sensors, it operates via UART communication. While its logic is 3.3[V] compatible, it is powered by the +5[V] rail through a dedicated converter to ensure its high-current infrared pulses do not destabilize the logic bus.
- Local Filtering and Stability (C4, C6, C7, C8, C9): To ensure measurement accuracy, each sensor is equipped with a 10 μ F decoupling capacitor placed in parallel with its power input. These capacitors act as local energy reservoirs, filtering high-frequency noise and voltage fluctuations caused by the rapid switching of the motor drivers.

ESP Block:

ESP32 is the heart of our board, serving as both a processor and a controller. It receives data from the sensors, processes it according to the embedded code, and, as a result, controls the motor drivers, which supply current to the inductors.

- Processing and Control Flow: The ESP32 acts as the central intelligence of the spinning top. It continuously polls data from the distance and orientation sensors, calculates the necessary corrective actions based on its internal algorithms, and sends control signals to the L9110S motor drivers. These drivers then regulate the electrical current delivered to the electromagnetic inductors to maintain the top's stability.
- Critical Protection (100 Ω Series Resistors): During the development of the Project B board, we encountered a significant hardware issue: when the ESP32 was connected to the board while powered by the 3S battery configuration (providing up to 12.6V), the controller would consistently burn out. To address this and protect the hardware, we integrated 100 Ω series resistors (R12–R27) on the GPIO lines. These resistors act as a protective buffer, limiting current surges from the high-voltage system and ensuring the ESP32 remains operational under the 3S power supply.
- Logical Level Correction (3.3k Ω Pull-up Resistors): In addition to the protection resistors, several 3.3k Ω pull-up resistors (R9, R10, R11) were added to the circuit. This was necessary because certain inputs and outputs on the ESP32 are hardware-limited and cannot receive or maintain a pure logical "1" on their own. The pull-up resistors force these lines to a stable high state, ensuring that communication with the sensors and interrupt signals function correctly.
- Power Filtering and Decoupling (C12, C3): To ensure the processor's power supply remains stable despite the high-current demands of the inductors, dedicated decoupling capacitors were placed at the power inputs:
 - C12 (10 μ F): Filters the 5[V] input line.
 - C3 (4.7 μ F): Stabilizes the 3.3[V] logic supply. These capacitors prevent the ESP32 from experiencing unexpected resets or brown-outs during high-load operations.

Motor Driver Block:

The Motor Driver block is the executive stage of the system, where logic commands are converted into physical magnetic force.

- Polarity Control (L9110S - U1, U2): The system utilizes two L9110S dual-channel drivers to control the electromagnetic coils. These drivers allow the ESP32 to flip the polarity of the electrical current. By rapidly changing the current's direction, the system can alternate the magnetic field to either attract or repulse the magnets on the spinning top. This high-speed switching is what will allow the top in the future to correct its angle, move through space, and maintain active balance.
- Project B Technical Constraint (Pin Conflict): During the implementation, a critical hardware conflict was identified regarding the second motor driver (U2).
 - The Issue: The control inputs for motor driver U2 were connected to GPIO 18, 5, 4, and 0.
 - The Failure: While the PCB physically supports the driver, GPIO 0 is a "strapping pin" required for the ESP32's bootloader sequence. Connecting it to the motor driver circuit interfered with the chip's ability to boot or flash firmware. Additionally, the internal state of these pins during startup made them unsuitable for reliable PWM control in this specific layout.
 - System Impact: Consequently, the hammer assembly associated with U2 was rendered non-functional. The project proceeded by utilizing the primary operational motor driver (U1), concentrating the control logic and stabilization on the active hammer system. This serves as a vital engineering lesson regarding the careful selection of GPIOs, particularly avoiding strapping pins for high-current peripheral drivers.
- High-Power Integration (+12[V] Rail): To generate the strong magnetic pull needed to move the top's mass, the functional motor driver is powered directly by the +12[V] rail. This separates the high-current switching of the coils from the sensitive logic of the processor, ensuring that the heavy power demands of the magnets do not destabilize the system.
- Local Filtering and Stability (C10, C11): Because the drivers switch high currents very quickly, they create electrical noise. To prevent this, 10 μ F decoupling capacitors (C10 and C11) are placed right next to the drivers. These act as local energy buffers that swallow voltage spikes and keep the power supply stable for the rest of the board.

LED Block:

The LED block is a central feature of the spinning top's interface. Beyond simple status indication, these LEDs are designed to function as a POV (Persistence of Vision) display. By flashing the LEDs at high speeds in precise synchronization with the top's rotation, the system creates the illusion of floating symbols, letters, and numbers in the air.

- GPIO Constraints and LED 8 sharing: Due to the finite number of available GPIO pins on the ESP32, we had to optimize our pin mapping. While LEDs 1 through 7 are connected to individual GPIOs, LED 8 shares its control line with the TF-Luna sensor. This dual connection is a direct result of the limited pin count, and it means that LED 8 and the TF-Luna sensor cannot operate simultaneously. For the POV display to show full symbols, the system must prioritize the LED sequence over the long-range sensor.
- Inverted Logic (Current Sinking): The LEDs are wired using Inverted Logic (Active-Low). In this configuration, the LED turns on when the ESP32 pin is pulled to '0' (Low). This method was chosen because it is electrically more efficient and easier for the ESP32 to sink current (pull current toward the ground) than to source current. This efficiency is vital when switching the LEDs thousands of times per second to generate sharp POV symbols.
- Series Resistors for Protection: Each LED is connected in series with a resistor. These resistors manage the voltage drop across the circuit and limit the current flowing through the LEDs. Without these resistors, the rapid switching required for displaying symbols would lead to excessive current drawing and burning out the LEDs or the ESP32 pins.

Test Points Block:

The Test Points block is an essential part of the board's development and maintenance processes. These points provide convenient access for electrical measurements to ensure the system's health and stability during operation.

- Power Rail Monitoring (TP1, TP2, TP3): These points allow for real-time verification of the system's three primary voltage levels, which is crucial for the 3S power architecture:
 - TP1 (3.3[V]): Used to monitor the logic supply for the ESP32 and sensors.
 - TP2 (5[V]): Monitors the supply for the distance sensors and intermediate power stages.
 - TP3 (V_Coil / 12[V]): In our 3S system, this is a critical point. It allows us to monitor the high-voltage rail that powers the electromagnetic coils, ensuring the DC/DC converter provides a stable 12[V] even under the heavy current loads required for stabilization.
- Reference Point (TP4): Serves as the system-wide Ground (GND) reference for all voltage measurements and oscilloscope probes.
- Non-Functional Test Points (TP5, TP6): While these test points appear in the schematic as battery/charging indicators, they are not relevant to this specific project. These nodes were part of an auxiliary system that is not utilized in the current 3S power configuration of Project B.

8.2. Layout

The PCB layout used in Project B is the same physical board developed during Project A. Its design was intentionally robust, allowing it to serve as a reliable foundation for the more advanced power requirements and control logic of the 3S system without needing a new revision of the hardware.

We included in this project book the highlights of the layout design process from Project A. For more details, read the Project A book.

1. Design Workflow: From Fusion360 to KiCad

To guarantee that the PCB fits perfectly within the spinning top's mechanical assembly, the design followed a Mechanical-First approach:

- Mechanical Constraints: The board outline, mounting holes, and critical component placements were first modeled in Fusion360. These constraints were exported as a .DXF file and imported into KiCad to serve as the master template.
- Electrical Integration: Using a custom-built symbol and footprint library, the schematic was translated into a Netlist (.NET), which defined all electrical connections (ratsnest) for the layout phase.

2. Layer Stackup and Routing Optimization

The board utilizes a 2-layer FR4 architecture (1.6mm thickness). This provides an optimal balance between manufacturing cost and signal integrity.

- Signal Integrity: We utilized 45-degree trace angles to prevent signal reflections and electromagnetic interference (EMI).
- Via Management: The use of vias was minimized to reduce parasitic inductance and capacitance, ensuring that high-speed signals between the ESP32 and sensors remain clean and stable.
- ESP32 Antenna Zone: A dedicated "Keep-out Area" was maintained under and around the ESP32 onboard antenna. This area is free of copper and traces to ensure optimal wireless communication.

3. Design Rules and Manufacturing Safety

To bridge the gap between digital design and physical fabrication, we applied strict Design Rules (DRC) based on professional manufacturing standards:

- Clearance: A minimum Copper-to-Hole clearance of 0.25mm was maintained to account for drill tolerances and prevent short circuits.
- Power Handling: Net classes were defined to differentiate between signal traces and high-power traces (12[V] rail), ensuring trace widths are sufficient for the current demands of the inductors.
- Solder Mask: A 0.38mm expansion and "Tent Vias" were used to protect the board from oxidation and soldering errors.

Scheme of the Layout:



4. Fabrication (Gerber Generation)

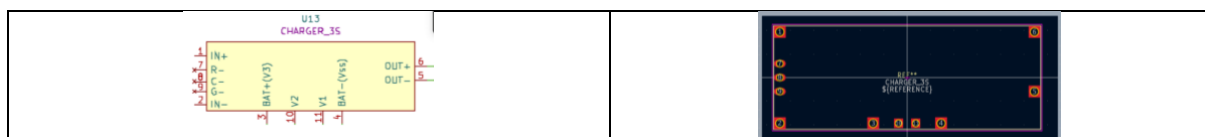
The final layout was exported into industry-standard Gerber files, representing the copper layers, solder masks, silkscreens, and drill locations. These files were sent to JLCPCB for professional fabrication.

Hardware Component Integration

While the core PCB layout remained consistent with the initial design, the integration of a 3S battery charger module necessitated updates to the schematic capture.

This required the creation of a custom component symbol within the KiCad environment, alongside the development of a corresponding footprint to ensure precise board-level assembly and connectivity.

These additions were seamlessly incorporated into the existing hardware model, preserving the integrity of the original PCB design while enabling the new power management functionality.



9. Software Overview

9.1 Android Mobile Application

9.1.1 Application Architecture and Workflow

The Android application uses a practical layered architecture inspired by the Model-View-Controller pattern. The design separates user-interface presentation, screen-level application logic, and network communication so that telemetry rendering, user commands, and connection management can evolve independently.



The View Layer (XML Layouts):

The XML layout files define the application screens, including controls, connection indicators, telemetry labels, and MPAndroidChart components used for real-time visualization.

Application and Controller Layer (Java Activities):

Android activities, such as `ConnectActivity`, `ChoiceActivity`, `VlActivity`, and `OperateActivity`, manage screen lifecycles, user actions, listener registration and register and updates to the Android main UI thread.

The Communication Layer- WebSocketManager:

A centralized `WebSocketManager`, implemented as a Singleton over the asynchronous `OkHttp` `WebSocket` pipeline, manages the bidirectional connection to the ESP32. It isolates networking from activities and distributes decoded telemetry and state updates to the currently active screen.

After launch, the application opens the connection screen and then transitions to the main selection interface. When a telemetry screen becomes active, it registers as the current listener. Incoming `WebSocket` frames are received on a background thread, decoded into application-level values, and forwarded to the UI thread for graphing, status display, and live monitoring.

9.1.2 Dual-Network Communication and Telemetry Pipeline

The ESP32 operates simultaneously in Access Point and Station modes, allowing the Android application and the laboratory computer to access different system services through separate network paths.

Access Point Interface - Android Application

The ESP32 creates a standalone Wi-Fi network for the Android device. The phone connects directly to this network and communicates with the firmware through a bidirectional WebSocket connection. The access-point interface uses the predictable gateway address 192.168.4.1, allowing the application to exchange control commands and binary telemetry without relying on an external router.

Station Interface - Computer and CSV Retrieval

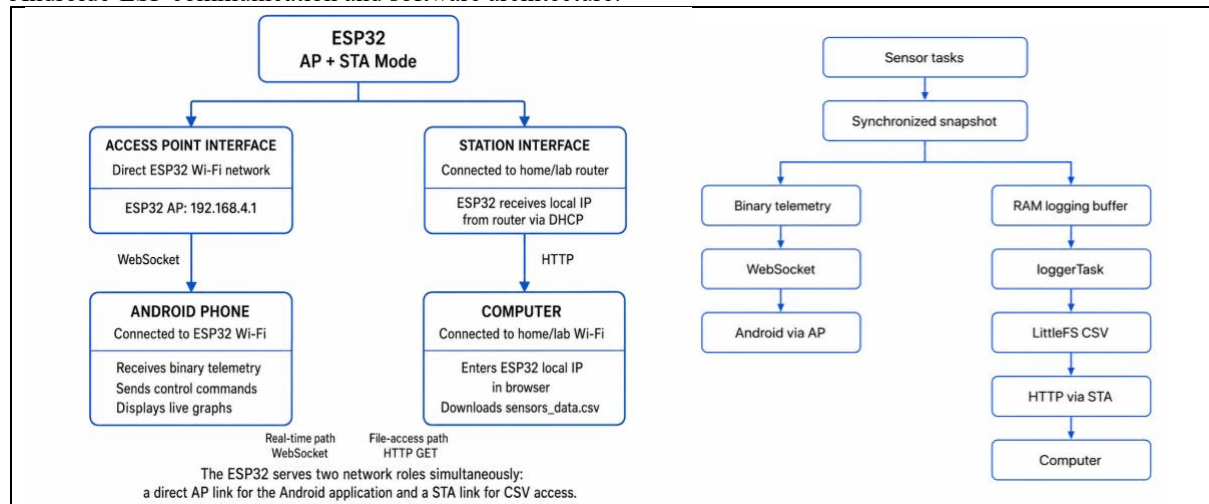
At the same time, the ESP32 connects as a station to the local home or laboratory Wi-Fi network. A computer connected to the same router can access the ESP32 through the local IP address assigned by the router. The computer enters this address in a web browser to request and download the latest CSV file stored in the ESP32 LittleFS memory.

The two interfaces serve different purposes:

- The AP interface provides a direct and predictable real-time link between the Android application and the ESP32.
- The STA interface provides browser-based access to the locally stored CSV file from a computer on the existing network.

Both interfaces are handled by the same ESP32 firmware, but the data paths remain functionally separate. Real-time telemetry is streamed to the Android application through WebSockets, while the CSV file is retrieved by the computer through an HTTP request.

Androide-ESP communication and software architecture:



The Android application is designed to receive high-rate telemetry from the ESP32 without overloading the UI thread. The firmware samples critical sensors at up to 200 [Hz], while the mobile client receives a throttled telemetry stream optimized for smooth visualization, typically 20 [Hz].

Hybrid WebSocket Protocol Routing:

The WebSocket connection used by the Android application carries two categories of messages:

- **Binary telemetry frames:** High-frequency sensor and state data are transmitted as a compact fixed-layout 99-byte packet. This avoids the parsing and allocation overhead of JSON during continuous streaming.
- **Text/JSON control frames:** Lower-frequency events and commands, such as `startSpinning`, `stopSpinning`, and on-demand status queries, remain in text form because they are easier to debug and occur far less frequently.

The Android client communicates with the ESP32 through a bidirectional WebSocket connection. The communication design separates high-rate telemetry from lower-rate control and state messages.



Binary Telemetry Frames

Continuous sensor and state data are transmitted in a compact fixed-layout binary packet. This avoids the repeated string conversion, allocation, and parsing overhead associated with high-frequency JSON messages.

Control and State Messages

Lower-frequency commands and state notifications remain in compact command or text-based formats because they occur less frequently and are easier to debug.

Direct ESP32-Android Connection

During standalone operation, the Android device connects directly to the ESP32 access point and communicates with the WebSocket server through the fixed gateway address `192.168.4.1/ws`.

Binary Parsing on Android

The Android application decodes telemetry using a `ByteBuffer` configured for little-endian byte order. Each field is extracted from a fixed offset matching the packed firmware structure.

This approach is faster and more deterministic than repeatedly deserializing text-based JSON objects.

Rate Separation

Internal sensing and logging continue at their required embedded rates, while telemetry sent to the Android interface is throttled to approximately 20 [Hz]. This preserves high-resolution local data without overloading the WebSocket connection or Android UI thread.

Memory-Optimized Graph Rendering

The previous approach for drawing a “moving” graph was inefficient: every time a message arrived from the spinning top, the application attempted either to create a new `ArrayList` object or to clear all the existing graph data and insert it again from scratch.

This caused dozens of small objects to be created every second, forcing Android’s garbage collector to work harder and continuously clean “garbage” from memory.

Since the garbage collector runs in the background, the application freezes for a fraction of a second. In a real-time graph, this appears as stuttering or display delays, making the spinning top look as if it is transmitting data in “waves” rather than as a continuous stream.

The final implementation uses a fixed-size sliding window of 250 samples. New points are appended to the graph while the oldest points are removed, allowing the same data structures to be reused continuously. This reduces allocation overhead, limits memory growth, and produces smoother real-time visualization.

From the device memory perspective, the data effectively “slides” through a fixed pipeline. No unnecessary new objects are created, and the garbage collector almost does not need to intervene.

Spin Counter Synchronization:

The spin counter is included in the same telemetry snapshot used by the esp32-side display logic. Therefore, the Android dashboard and the POV mechanism refer to the same firmware-generated rotation state, even though they operate at different update rates.

9.1.3 Android Visualization and Connection Optimization

The final Android implementation was optimized to prevent network traffic and graph rendering from overloading the mobile UI thread.

Only the currently active activity receives live telemetry updates. The graphing layer uses a fixed-size sliding window and reuses existing data structures instead of rebuilding the complete dataset for every received frame. UI updates are performed at the controlled telemetry rate rather than at the full embedded sampling rate.

The application also exposes the WebSocket connection state explicitly. Connection loss is reported to the user, and reconnection is initiated from the connection screen instead of being hidden behind uncontrolled automatic retry loops. This improves fault visibility and makes recovery more predictable during hardware testing.

9.1.4 Application Interface and Operational Guide

Step 1: Hardware Activation and Calibration:

Connect the spinning top to its power source. During startup, the LED array indicates initialization. When the inertial sensor calibration routine finishes, the system is ready for communication and testing.

Step 2: Network Association:

On the Android device, connect to the ESP32-Top Wi-Fi access point using the configured password (12345678). If Android warns that the network has no internet access, select the option to remain connected. During testing, disabling mobile data can prevent the operating system from routing traffic away from the ESP32 network.

Step 3: Application Launch and Synchronization:

Open the application and connect to the WebSocket server. After successful binding, the connection indicator should show that the WebSocket is open and ready.

Step 4: Real-Time Telemetry Verification:

Open a monitoring screen, such as `VlActivity`. The distance-sensor channels should stream to the graph window. Moving an object near the sensors should create corresponding distance changes in the plotted values.

Step 5: Control Execution and Actuation:

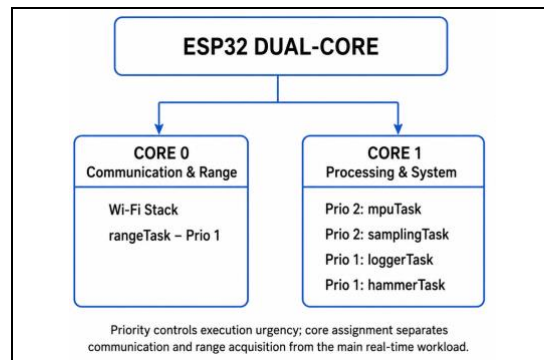
Open `OperateActivity` to send motion or hammer-control commands. The command is transmitted as a compact binary control packet. The firmware updates its internal state and returns periodic state confirmations so that the mobile interface reflects the current hardware mode.

Debugging Support:

LOGCAT messages were added in Android Studio to monitor connection status, received packets, activity transitions, parsing errors, and command responses during real-time testing.

9.2 Embedded Firmware Component (ESP32)

9.2.1 FreeRTOS Task Architecture and Core Assignment



To meet real-time constraints, including sensor acquisition, telemetry preparation, logging, actuation, and communication under different timing requirements., the ESP32 firmware uses a FreeRTOS-based multitasking design.

Task Allocation and Core Pinning (Constraint-Driven Design):

Initial testing showed that placing all high-frequency operations on the same core increased timing jitter. The final design separates tasks according to their timing sensitivity and hardware dependencies.

Core 1- Real-Time Processing and System Tasks

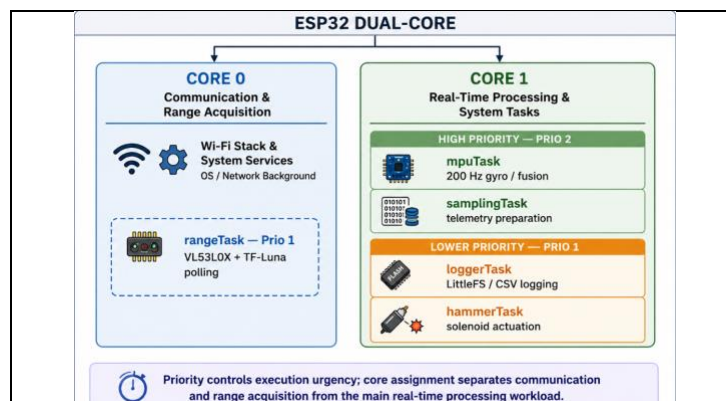
Core 1 runs the high-priority `mpuTask` (200[Hz]/5[ms] interval) and `samplingTask` (50[Hz]/20[ms] interval), together with the lower-priority `loggerTask` and `hammerTask`.

The `mpuTask` and `samplingTask` are assigned Priority 2 because they perform high-frequency angle processing and telemetry preparation. The `rangeTask`, `loggerTask`, and `hammerTask` are assigned Priority 1 because they operate at lower rates or depend on slower hardware and storage operations.

This architecture does not create an absolute separation between real-time and background work. Instead, it combines core pinning and priority-based preemption so that the high-frequency tasks can continue running while lower-priority operations share the remaining processing time.

Core 0 - Communication and Range Acquisition

The Wi-Fi stack naturally runs on Core 0. Slower distance-sensor polling, such as `rangeTask` at approximately 20[ms] intervals, can also be assigned to Core 0 at a lower priority to reduce load on the real-time core. Care must still be taken to avoid blocking calls that could interfere with Wi-Fi timing.



This allocation, with the selected tasks pinned to Core 1, enables high-frequency tasks to be isolate from Core 0, which is responsible for Wi-Fi handling, without overloading either Core 1 or Core 0. File logging and hammer actuation run at lower priority. This ensures that flash writes and mechanical-control actions do not delay the high-rate angle and sampling tasks.

9.2.2 Shared-State Synchronization and Snapshot Distribution

Multiple firmware tasks update different fields of the shared `dataAll` structure. Although individual scalar writes may be atomic, the complete multi-field system state can still be read while only part of an update has been completed. This may produce a torn or internally inconsistent telemetry snapshot.

To avoid this condition, the software uses a hybrid writer-lock and sequence-counter mechanism.

Writer Protection: Tasks that modify the shared state enter a short protected update section using `dataWriteBegin()` and `dataWriteEnd()`.

A FreeRTOS synchronization primitive prevents two writers from modifying the structure at the same time.

Optimistic Snapshot Reading: The `samplingTask` reads the structure without holding the writer lock for the entire copy (avoids blocking the real-time writers).

It first reads the sequence counter, copies `dataAll` into a local snapshot, and then reads the sequence counter again.

A snapshot is accepted only when (1) and (2) occur:

- (1) the sequence value is even, indicating that no write was active.
- (2) the sequence value remains unchanged before and after the copy.

If either condition fails, the snapshot is discarded and read again.

This mechanism allows `samplingTask` to obtain an internally consistent system-state snapshot without holding the writer lock during the full copy, which may introduce a jitter to the POV display caused by unpredictable latency that slows down the telemetry stream that could happen if the reader holds writer's mutex.

The same accepted snapshot is then used for telemetry packaging and CSV buffering, ensuring that the streamed and recorded representations are derived from the same firmware state.

Snapshot Consistency via Seqlock

The Seqlock mechanism is critical for guaranteeing the integrity of the extracted data without blocking the system. The seqlock is based on an even/odd sequence counter.

Before initiating a read by `samplingTask`, the reading task inspects the sequence counter.

If the counter is odd (`s1 & 1u`), it indicates an active write operation is currently in progress, prompting the reader to briefly yield and wait.

Once the counter is *even*, the reader performs an “optimistic copy” of the data structure (from `dataAll` to local snapshot).

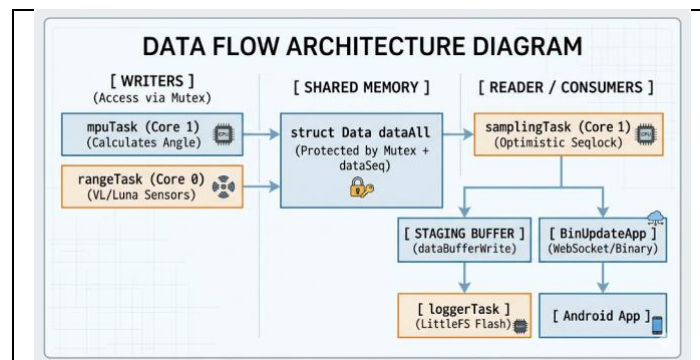
Upon completing the data copy process, the reading task compares the initial and final states of the sequence counter (`s1 != s2`).

If the counter remains unchanged, it validates that no concurrent write occurred, guaranteeing a perfectly consistent and intact snapshot.

Conversely, if the counter values differ, it indicates that the data was modified mid-read, resulting in a corrupted snapshot (Data Tearing).

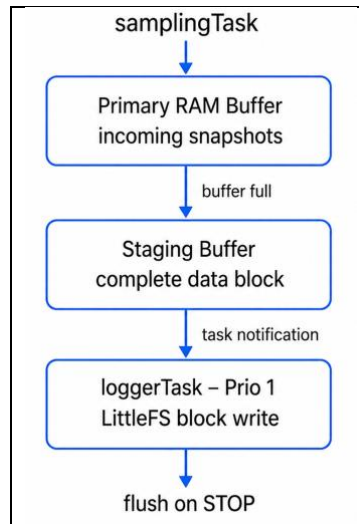
In this event, the reader immediately discards the invalid data and initiates a new read cycle.

Ultimately, while the `Mutex` prevents write-write conflicts, the Seqlock is the core mechanism that ensures the reader obtains a consistent, atomic snapshot (no partial or torn telemetry packets) without introducing blocking latency to the real-time tasks and prevents.



9.2.3 Data Logging Optimization and Memory Buffering

The logging path was redesigned so that LittleFS write and flush operations would not interfere with time-sensitive sensing and POV display execution. The implementation separates snapshot collection from flash-memory access through RAM buffering, a lower-priority logging task, block-based writes, and deferred flushing.



Layer 1: RAM Buffering and Asynchronous Staging

Accepted telemetry snapshots are first accumulated in a primary RAM buffer, `dataBuffer`. When the buffer reaches its configured capacity of `BUFFER_SIZE = 256` samples, the `samplingTask` copies the completed block into a dedicated staging buffer, `dataBufferWrite`, using `memcpy()`.

After the transfer, the `samplingTask` sends a FreeRTOS task notification to `loggerTask` and immediately resumes collecting new snapshots into the primary buffer. Meanwhile, the lower-priority `loggerTask` writes the staged block to LittleFS.

This mechanism decouples high-frequency data production from slower flash-memory access, allowing sampling to continue while the completed block is processed in the background.

Layer 2: Block-Based File Writing

The `loggerTask` writes complete blocks using `file.write()` rather than performing a separate file-system operation for every individual sample.

Writing 256 samples in one block reduces the number of costly file-system calls, lowers write overhead, and preserves complete telemetry records. The internal consistency of each record is established earlier, during synchronized snapshot creation; block-based writing is responsible for storage efficiency and timing isolation rather than for preventing data tearing.

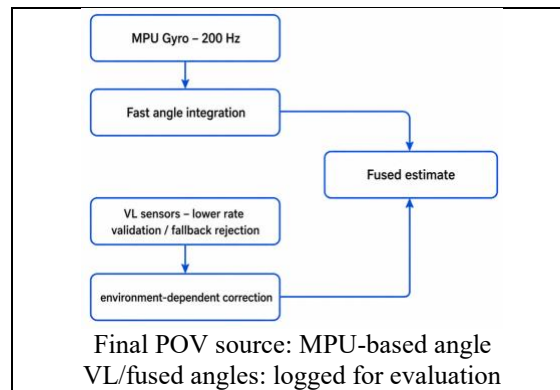
Layer 3: Deferred Flush Strategy

The firmware distinguishes between writing data into the file with `file.write()` and forcing storage synchronization with `file.flush()`.

During active rotation, completed data blocks are written without flushing after every block. The more expensive flush operation is deferred until the system enters the stopped state. This reduces flash-related latency spikes and minimizes interference with high-frequency sensing and POV rendering.

Together, RAM staging, background block writing, and deferred flushing allow high-resolution CSV logging to operate alongside real-time sensing and display tasks without placing slow file-system operations directly in the sampling path.

9.2.4 Sensor Fusion and Attempt for Drift Correction via Spatial Vector Integration



The system does not currently use an absolute magnetic reference (due to mistake in the PCB designed in Project A). Therefore, the firmware estimates orientation by combining high-rate gyroscope integration with lower-rate geometric correction from the distance sensors.

Gyroscope Integration:

The primary angle estimate is calculated by integrating the MPU6050 Z-axis angular velocity at 200 [Hz]. This provides smooth short-term motion tracking, but it is susceptible to accumulated drift over time.

Geometric Correction from VL53L0X Sensors:

Three VL53L0X distance sensors are positioned 120 degrees apart.

Their distance readings can be interpreted as a spatial profile around the spinning top.

A geometric vector can be estimated by projecting each reading onto its known sensor direction and calculating the resulting vector angle.

A clearer mathematical representation is:

$$x = \sum d_i \cos(\theta_i), \quad y = \sum d_i \sin(\theta_i), \quad \varphi = \text{atan2}(y, x)$$

where d_i is the distance reading and θ_i is the fixed angular position of each sensor.

The values of x , y and φ describe the spatial centroid of distance readings, treating the sensor array as rotating LIDAR system. By projecting the radial distances onto a fixed 2D coordinate system, we estimate a dynamic special vector that reflects the top's environmental proximity profile.

Fusion Behavior:

The `updateAngle` function merges these data streams, and uses the geometric vector as a soft correction term (from the VLs) rather than a hard reset (absolute magnet).

The system continuously generates a dynamic reference vector derived from the spatial distance centroid (from VLs), and in principle, can reduce angular drift of MPU angle.

This vector acts as an external spatial anchor, allowing the system to perform real-time orientation correction relative to the surrounding environment and should allows the high-rate gyro angle to remain smooth while slowly reducing long-term drift.

Experimental Limitations:

Although the firmware supports drift correction via the VL sensor array (2% correction), experimental data showed frequent VL fallback events, in which at least one distance channel returns a default or invalid value. Also, these sensors operate at a lower rate than the MPU, the computed vector depends on the geometry of the surrounding environment and symmetric or slowly changing distance profiles may not provide a unique angular reference.

Since the fused angle logic only operates when valid sensor data is present, the significant prevalence of these fallback events creates extended periods without angular correction, reduces the effectiveness of drift correction and allow the gyro-integrated angle to keep drifting.

Therefore, the final POV display relies primarily on the MPU-integrated angle. The VL-derived angle and fused estimate are retained in the CSV output for analysis, but they do not provide a reliable long-term replacement for an absolute phase reference.

9.2.5 Binary Telemetry Protocol and Bandwidth Efficiency

To stream high-frequency telemetry efficiently, our software design uses a custom binary packet instead of continuous JSON messages. The packet is a packed 99-byte `masterPacket` structure that contains sensor values, angles, counters, timestamps, and system-state fields, encoded with `__attribute__((packed))`. Text-based formats (like JSON) were avoided because their serialization and string-parsing overhead would consume excessive CPU cycles and increase latency, potentially causing "message queuing" errors.

Binary telemetry improves performance because each variable is stored in its native fixed-size representation. For example, a `uint16_t` always occupies 2 bytes, while the same value in JSON requires character conversion, delimiters, and key names.

This reduces CPU formatting overhead, network payload size, file size, and flash-write load.

On Android, the binary payload is decoded with a `LITTLE_ENDIAN ByteBuffer` and fixed offsets so that the parsing occurs directly to the memory.

Since parsing happens away from the UI thread and does not require JSON allocation, it avoids costly string allocations and JSON decoding cycles, allowing the UI thread to update the real-time graphs with minimal overhead so that the graphs remain more responsive during continuous streaming and the WebSocket connection maintain the high-speed update rate without saturating the Wi-Fi bandwidth.

9.2.6 Telemetry Throttling and UI Synchronization:

Although the binary protocol significantly reduces packet size and transmission overhead, transmitting and rendering every sample on Android is unnecessary and can harm performance. Our implementation uses adaptive data throttling (downsampling) for the telemetry stream to the app.

Sending telemetry at the raw sampling rate of 200[Hz] is counterproductive for two reasons:

1. **Rendering Limitations:** The Android UI thread cannot render graph updates at 200[Hz] without incurring significant CPU overhead and UI stutters.
2. **Network/Stack Efficiency:** High-frequency messaging increases context switching overhead between the real-time sensor tasks and the Wi-Fi communication stack, potentially starving time-critical operations. Consequently, the telemetry is throttled to a refresh rate optimized for human visual perception and UI stability (e.g., 20-50[Hz]). This ensures that the mobile application remains responsive while maintaining a consistent and smooth data flow, balancing the firmware's high-precision sampling with the UI's limited rendering capacity.

In our implementation, telemetry and `spinCount` are sent to application every 50[ms] (20[Hz]) and operational commands response (for moving top/hammer indication) are sent to application every 600[ms].

The telemetry stream is therefore downsampled for the user interface:

- The firmware continues high-rate internal sampling and logging.
- The Android application receives telemetry approximately every 50 [ms], corresponding to 20[Hz].
- Operational state confirmations, such as top/hammer movement indicators, are sent at a lower rate, approximately every 600[ms], because these values do not require high-frequency graph updates.

This split balances high-precision embedded sampling with the Android UI rendering capacity and prevents WebSocket queue growth.

9.2.7 Sensor Porting and I2C Compatibility:

Following a hardware revision, the MPU chip exhibited a non-standard I2C address (0x70 instead of 0x68), the communication subsystem was ported to the flexible MPU6050 `_tockn` library (instead of Adafruit). This simplified offset calibration and improved compatibility with the actual I2C configuration used in the project. If the system uses an I2C multiplexer or additional device mapped at address 0x70, this should be described as part of the I2C routing architecture rather than as the native MPU6050 address.

9.2.8 POV Display Mechanism

The POV display uses the persistence-of-vision effect: when the top rotates quickly, short LED pulses appear to the human eye as a stable visual pattern.

The firmware converts the current spin count into a digit and lights selected LEDs only when the estimated angular position falls inside the segment window assigned to each part of the digit.

The mechanism operates in four stages:

- Angle estimation: The firmware continuously estimates the current angular position using the gyroscope and, when possible, distance-sensor correction.
- Digit selection: The digit to display is derived from the spin counter, for example by advancing after a fixed number of completed rotations.
- Segment-window matching: Each digit is represented as a set of vertical LED-column patterns. The firmware checks whether the current angle falls inside the active display window for that column.
- Timed LED activation: Only the required LEDs are turned on for a very short interval. As the top rotates, these timed pulses combine visually into a readable stationary-looking digit.

Because the display depends directly on timing and angle accuracy, any blocking operation, excessive WebSocket traffic, or flash-write stall can create visible artifacts. This is why task isolation, buffered logging, and telemetry throttling are essential parts of the software design.

Currently, the POV display uses the MPU-based angle because it provides the most reliable high-rate phase estimate during rotation. Alternative VL-derived and fused angles are retained in the CSV file for comparison and analysis. By changing the displayed angle we can easily alternate the MPU-based angle to the Fused angle.

A timing-compensation term is applied to reduce visible phase lag when the spin speed changes. This improves short-term display alignment but does not remove accumulated gyroscope drift. A future design should therefore include a stronger absolute phase reference.

10. Testing and Performance

10.1. Independent Testing

Prior to full system integration, each hardware and software component of the spinning-top underwent independent validation. In this way we isolate individual modules, verify their core functionality, and identify performance limits that could later impact the complete real-time system.

Electrical Testing:

In Project B, the testing phase was expanded to address the challenges of the new 3S power architecture and the high-current demands of the electromagnetic coils. To ensure the safety of the sensitive electronic components, the assembly of Project B followed a strict incremental protocol. At each stage, we performed a "No-Load" voltage test, followed by a "Load Test" (using a power resistor), before proceeding to the next block.

1. **Individual Battery Cell Testing:** Each Lithium-Ion cell was tested independently to verify its nominal voltage and health.
2. **Series Connection:** The three cells were connected in series to achieve the required voltage potential.
3. **3S charge controller:** The 3S voltage controller was connected to the series pack. We verified that the controller correctly managed the combined voltage and provided a stable output while protecting against over-discharge or short circuits.
4. **Power Converter Stage (DC/DC):** The converters were connected to the 3S pack. We adjusted and verified the stable outputs of 12[V] (for coils) and 5[V] (for sensors/logic).
5. **ESP32 Microcontroller:** Once the power rails were confirmed stable under load, the ESP32 was mounted. We verified that it received a clean 3.3[V] supply and initiated its boot sequence.
6. **Sensor Array:** The MPU and distance sensors were connected. We monitored the I2C bus to ensure no voltage drops occurred during data transmission.
7. **Motor Drivers and Hammer Assembly:** The L9110S drivers and electromagnetic coils (hammers) were the final stage. We tested the high-current switching of the coils, ensuring that the magnetic pulses did not destabilize the rest of the system.

The Simulations section provide a detailed examination of the tests above.

Software Testing:

The initial software testing phase for Project B focused on test verifying that the core logic from Project A was compatible with the upgraded mechanical and electrical assembly. However, this phase revealed a critical hardware-software interaction failure:

- The Incident: Following the mechanical redesign, the independent operational code from Project A was uploaded to the ESP32 to test the indented basic actuation of each component. Upon execution, the ESP32 microcontroller suffered catastrophic failure (burned out).
- Root Cause Analysis: In Project A, the motor drivers and coils operated at a significantly lower voltage. With the transition to the 12[V] (3S) power rail in Project B, the peak currents skyrocketed. During high-speed switching, inductive kickback and transient voltage spikes from the coils breached the isolation of the driver inputs, feeding back directly into the ESP32 GPIO pins. Lacking sufficient inline current limiting, the MCU (microcontroller unit) pins were overstressed, leading to internal shorts.
- The Engineering Fix: To protect the MCU from these destructive transients without spinning a new PCB layout, we modified the existing board by adding inline current-limiting 100Ω protection resistors between the ESP32 GPIO pins and the L9110S driver control inputs. Subsequent software tests confirmed that these resistors successfully dampened signal-line spikes and clamped the current to safe, non-destructive levels during aggressive active stabilization.

1. Sensors and Rotation Tracking:

Initial testing focused on the perception modules. The MPU6050 was evaluated for angular velocity accuracy, integration logic, and stability under both static and dynamic conditions. Concurrently, the VL53L0X distance sensors were assessed to confirm reliable measurements, fallback behaviors, and realistic sampling rates.

Following sensor validation, the rotation-counting mechanism was isolated to ensure it accurately detected full rotations. The logic was tested across varying rotational speeds and directions, confirming that the internal counter updated consistently.

Establishing this reliability early was crucial before coupling the counter with time-sensitive tasks like sensor fusion and POV display timing.

2. Actuation and Visual Display:

The motor/ hammer mechanisms were examined for response latency, movement modes, and overall mechanical stability. A key focus was determining the fastest reliable actuation rate for the hammer without inducing mechanical erraticism. Furthermore, because the power delivery was upgraded to a battery-based system, the hammer's performance was specifically re-evaluated to account for the new power constraints.

For the visual interface, the Persistence of Vision (POV) and LED control logic were verified using simulated angular inputs. By feeding predefined angles and digit values into the display pipeline, we confirmed correct segment activation and precise angular windowing before running the code on the physically rotating device.

3. Data Communication, Logging, and Visualization

The network layer was validated by establishing a bidirectional WebSocket connection between the ESP32 and the Android client. While early debugging relied on readable JSON messages, the system was transitioned to compact binary payloads to optimize real-time performance.

Tests confirmed stable connections, reliable transmission of 4-byte control commands, and accurate decoding of the `masterPacket` telemetry structure on the mobile device.

Similarly, the CSV logging mechanism was tested in isolation to ensure that file I/O operations would not block or interfere with the high-frequency sensor sampling and telemetry pipelines.

Finally, the Android application's graphing engine was optimized. Initial versions experienced UI stuttering and memory pressure due to frequent data structure reallocation. By implementing a fixed-size sliding window and reusing memory allocations, we eliminated garbage collection bottlenecks, achieving smooth, real-time visualization of the system's dynamics.

The Software-Overview section provide a detailed information.

10.2. Integration Test

On Project A the integration phase was focusing on validating the connections between components. However, in Project B we focus on evaluation the concurrent execution of all hardware and software modules under realistic operating conditions.

The fundamental challenge lay in efficiently coordinating sensors, display logic, motor control, wireless communication, and data logging within strict real-time constraints, ensuring they operate continuously without inducing blocking states or systemic instability.

During the integration, we examined how the sensors, display logic, motor control, wireless communication, Android application, and data logging operate together on 12[V] system.

Electrical Testing:

Once system stabilization was achieved, and after verifying that each component functioned correctly both independently and synchronously within the integrated system, we proceeded to evaluate the noise levels across the device (after adding the resistors).

Signal-to-Noise Ratio Test

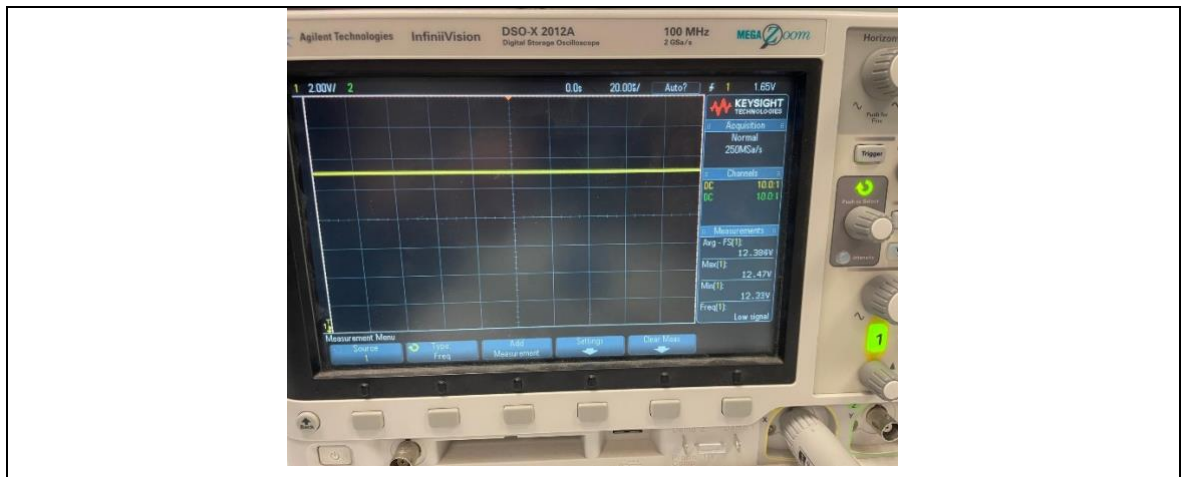
To evaluate the quality of the DC voltage output, we measured the Signal-to-Noise ratio (SNR) under the following conditions:

1. Charge controller connected to a charging source:
 - 1.1 DC-DC converter is disconnected from board, in cases with and without a resistive load.
2. Charge controller is disconnected from the charging source:
 - 1.1 DC-DC converter is connected to board, in cases with and without a resistive load.
 - 1.2 DC-DC converter is disconnected from board, in cases with and without a resistive load.

MARKS:

1. In all tests, batteries were connected to board.
2. When DC-DC converter is connected, its output is tuned to 11.5[V], denotes V_{coil} .
3. When resistive load is connected, loads the output of the previous unit.
4. SNR calculation corresponds the formula $SNR_{dB} = 10 \log_{10} \left(\frac{P_{signal}}{P_{noise}} \right) = 20 \log_{10} \left(\frac{V_{signal}}{V_{noise}} \right)$.

Initially, we connected the three batteries in series under no-load conditions.

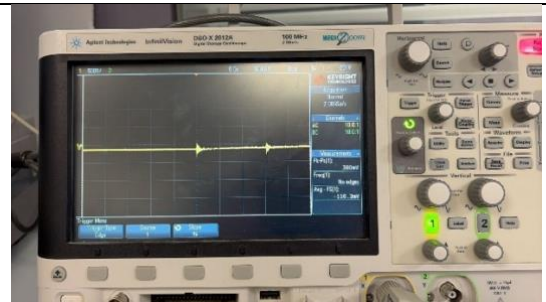
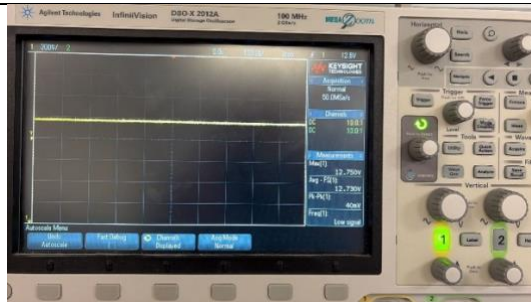


Charge controller was connected to the charging source without DC-DC Converter:

- No Load:

The measured DC voltage was:
 $V_{DC} \approx 12.730[V]$

The measured average voltage noise was:
 $V_{noise,avg} \approx 10.1[mV]$
 The measured Peak-to-Peak voltage was:
 $V_{P2P} \approx 60[mV]$

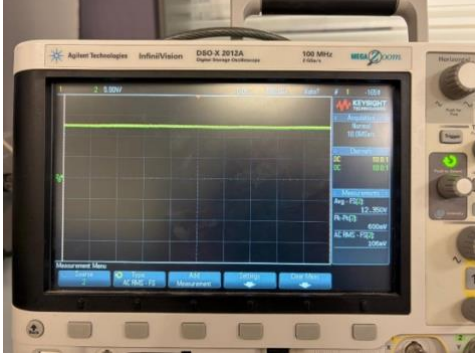


calculation of the SNR derives:

$$SNR_{dB} = 20 \log_{10} \left(\frac{12.730[V]}{60[mV] - 10.1[mV]} \right) \approx 28.134 [dB]$$

It indicates that the noise is 3.92% of the DC signal ($\frac{1}{\frac{28.344}{10} - 20} \times 100\% = 3.92\%$).

We now want to load the charge controller output with a resistor. The input resistances of the voltage converters are 500 k Ω , so we chose a resistor with a relatively high resistance. The resistor with the highest resistance available to us has a value of 15 k Ω . The resistor current was 0.83mA.

The measured DC voltage was: $V_{DC} \approx 12.35[V]$	The measured average voltage noise was: $V_{noise,avg} \approx -30[mV]$ The measured Peak-to-Peak voltage was: $V_{P2P} \approx 380[mV]$
	
calculation of the SNR derives: $SNR_{dB} = 20 \log_{10} \left(\frac{12.35[V]}{380[mV] - 30[mV]} \right) \approx 10.95 [dB]$ This corresponds to 28.34% noise relative to the DC signal ($\frac{1}{10^{10.95/20}} \times 100\% = 28.34\%$).	

We connected the 5[V] DC/DC converter, with no load at the output.

The measured DC voltage was: $V_{DC} \approx 4.801$	The measured average voltage noise was: $V_{noise,avg} \approx -31.2[mV],$ The measured Peak-to-Peak voltage was: $V_{P2P} \approx 80[mV]$
	
calculation of the SNR derives: $SNR_{dB} = 20 \log_{10} \left(\frac{4.801[V]}{80[mV] - 31.2[mV]} \right) \approx 19.858 [dB]$ This corresponds to 10.16% noise relative to the DC signal ($\frac{1}{10^{19.858/20}} \times 100\% = 10.16\%$).	

Due to the campus closure during the Israel-Iran war (February-April 2026), the laboratory tests could not be completed.

Software Testing:

1. Real-Time Task Synchronization

A significant complexity during integration was managing the synchronization across parallel FreeRTOS tasks. The system architecture encompasses multiple high-frequency threads, including MPU and range-sensor sampling, telemetry generation, WebSocket transmission, and local file logging. Because these tasks operate concurrently on shared data structures, robust synchronization mechanisms were imperative. We validated locking protocols and snapshot-based access to prevent race conditions, ensuring that both the telemetry streamed to the Android client and the data written to the local CSV file accurately reflected a coherent and stable system state.

2. POV Display and Spatial Tracking Integration

The Persistence of Vision (POV) display and spin-count algorithms were tested jointly to verify visual and logical consistency.

Testing confirmed that the LED digit activation was tightly synchronized with the estimated angular position of the spinning top, and that the spin counter updated reliably during rapid rotations. To mitigate drift and enhance display stability, we assessed and fused multiple spatial inputs, combining MPU-based inertial estimation with distance-sensor-based corrections.

3. Data Pipeline and Visualization Optimization

End-to-end testing exposed initial performance bottlenecks within the communication and visualization pipelines. Early implementation utilizing verbose JSON payloads and unrestrained graph updates suffered from excessive communication overhead and Android UI stuttering. To deal with this issue, the data stream was transitioned to a compact binary protocol.

In addition, an optimization on the client-side visualization has also been implemented via controlled UI throttling and the implementation of a fixed-size sliding window for real-time graphing.

Crucially, The CSV logging and optimized Android graphing mechanism were tested together to verify that both represent the same system behavior to guarantee system-wide consistency. By decoupling the two processes, the CSV logger preserved high-resolution, unthrottled data for accurate offline analysis, while the Android client processed a reduced stream suitable for smooth real-time monitoring.

This allowed the ESP32 to maintain high-frequency sampling and local recording without overloading the mobile device's rendering thread.

The Software Overview section provide a detailed information.

11. Results, Conclusions and Optional Solutions

This chapter summarizes the performance of Project B, the integration of the 3S power system, and the critical engineering lessons learned during the transition from the previous hardware iteration.

11.1 Power System & Reliability

The 3S Challenge

The transition to a 3S battery configuration provided the necessary power for the system but introduced significant maintenance and reliability challenges:

- **BMS (Battery Management System) Reliability Issues:** The battery management module proved to be overly sensitive. It frequently entered a "lockout" state, requiring a physical disconnection and reconnection of the batteries to resume operation. In addition, the module was easily damaged.
 - **Conclusion:** The current setup makes it impossible to keep the system sealed for long periods. Future iterations must utilize a more robust BMS or a mechanical design that allows for an external hardware reset without dismantling the cone.
- **Wiring Density and Integration:** Due to the added batteries and the removal of the original holder, the internal space within the cone became extremely congested. Initial mounting with adhesive or clay proved insufficient against the vibrations of high-speed rotation.
 - **Recommendation:** Future designs should include a secondary "Shield" PCB or dedicated internal wiring channels to prevent component shifting and wire fatigue.
- **Electrical Noise (SNR):** Lab tests revealed significant signal-to-noise ratio issues. This was caused by the proximity of the 12[V] power lines (which drive the high-current coils) to the sensitive data lines of the sensors.
 - **Proposed Fix:** Physical isolation is required - routing power cables on one side of the cone and signal logic on the opposite side to minimize electromagnetic interference (EMI).

11.2 Mechanical Balancing & Material Selection

- **Polymer Clay:** This proved to be an excellent balancing material. It offered superior adhesion, did not dry out (unlike modeling dough), and maintained a cleaner workspace compared to standard plasticine.
- **Random Equilibrium Test:** The balancing procedure was a success. The top consistently came to a halt in random orientations, proving that the Center of Mass (CM) was successfully aligned with the axis of rotation.
- **Spinning duration:** Under motorized drill acceleration, the top achieves an average total spin time of around 30 seconds, most of them stable.
- **Surface Interaction:**
 - **Optimal:** Smooth floors and polished wooden tables provided the best results.
 - **Sub-optimal:** Glass (causing excessive vibration/slippage) and cardboard (which absorbed too much rotational energy).
- **Spinning Tip:** The use of a metallic tip proved very effective, offering the best balance between low friction and durability during high-speed operation.

11.3 Actuation and Hardware Evolution

- **Hammer Force:** Operating at 12[V] resulted in a massive increase in electromagnetic torque. The "hammers" now strike with significantly more force, allowing for aggressive angular corrections that were impossible in Project A.
- **PCB Robustness:** A major engineering success was the ability of the original Project A PCB to handle the 3S upgrade. Aside from adding protection resistors for the ESP32, the board handled the increased currents perfectly, validating our initial "robust design" approach.

11.4 Firmware and I/O Constraints

- GPIO Scarcity: The ESP32 has reached its physical I/O limit. This currently prevents the addition of a third hammer or additional sensors like a photoresistor.
- The Strapping Pin Lesson: We learned that connecting high-load components to GPIO 0 interferes with the boot process.
 - **Conclusion**: Future revisions must incorporate an I/O Expander to offload non-timing-critical tasks (like LEDs) and free up the native pins for high-speed sensing.

11.5 Hardware Fixes

- TF-Luna Wiring Correction: During the integration of the LiDAR sensor, it was discovered that the original PCB layout (from Project A) had reversed the TX/RX lines.
 - **The Fix**: This required a manual wire crossover during the sensor connection. This highlights the importance of "pin-to-pin" verification during the design phase, as documented in Project A error log.
- Shared Resource Management (LiDAR vs. LED 8): Due to the limited GPIO count, the TF-Luna and LED #8 were forced to share a communication line.

11.6 Frequency Constraints and Sampling-Rate Tradeoffs

A major design consideration in the system was that each component operates under different frequency limitations. Therefore, the system was designed as a multi-rate real-time architecture.

The POV LED rendering loop operates at approximately 1[ms] (1000[Hz]), because the displayed digit must be updated with very fine timing resolution during rotation.

The MPU6050 was sampled every 5 [ms], corresponding approximately 200 [Hz], to provide sufficiently fast angular-velocity updates for continuous angle integration and POV LED synchronization. A slower MPU update rate would increase angular drift and reduce the accuracy of the displayed digit position.

The VL53L0X distance sensors are hardware-limited by their ranging time and practical I2C polling overhead. They were sampled in fast ranging mode of 20[ms] (50[Hz]), allowing reliable distance measurements without overloading the bus or producing excessive fallback values.

The TF-Luna Sensor provides range data at a limited practical update rate around 10[ms] (100[Hz]) in fast mode, and useful for correction and feedback, and is used as an additional distance reference rather than as the primary high-frequency angle source.

Both sensor types cannot replace the high-frequency MPU-based timing required for POV rendering.

At the communication and visualization levels, the data rate had to be reduced even further.

Although internal tasks can run at 200 [Hz] or even 1000 [Hz], sending every internal update to the Android application would overload the WebSocket channel and the mobile UI.

The Android phone display and graph rendering pipeline, tested on a Galaxy A52 running Android 14, cannot meaningfully visualize hundreds of updates per second.

On this mobile, display refreshes at approximately 60–90 [Hz], and useful human visual feedback is typically around 30–60 [Hz].

Therefore, WebSocket streaming and graph updates were intentionally throttled.

Although synchronized sensor snapshots are generated at approximately 20 [ms] (50 [Hz]), the WebSocket telemetry stream is throttled to approximately 50 [ms] (20 [Hz]) before being sent to the Android application.

This separation allowed the firmware to keep high-frequency real-time logic local, while sending the Android application only the amount of data needed for graph smoothness and readable visualization (this reduction of bandwidth prevented UI flooding without losing meaningful visual information).

11.7 Real-Time Software Pipeline Optimization

The final software architecture was developed through a series of communication, synchronization, storage, and visualization optimizations. Together, these changes allowed high-rate embedded processing to coexist with responsive Android monitoring, full-resolution CSV logging, and time-sensitive POV execution.

During the early stages of integration, sensor data was transmitted too frequently to the Android application. Even when the ESP32 was able to collect data at high rates, sending every update directly through the WebSocket created unnecessary bandwidth pressure and increased the processing load on the mobile client.

To solve this, the WebSocket transmission rate was reduced and separated from the internal sensor sampling rate. The ESP32 can continue performing fast internal sampling and control tasks, while the Android application receives data at a controlled visualization rate. This preserves real-time internal behavior without flooding the communication channel or overwhelming the user interface.

- Communication Optimization: JSON to Binary Packet

The initial communication protocol was based on JSON messages containing multiple high-frequency sensor fields. Although JSON was convenient for debugging and development, it introduced significant overhead due to repeated string parsing, large message size, and frequent memory allocations. As the telemetry rate increased, this approach became inefficient and started to overload both the ESP32 communication layer and the Android application.

To reduce this overhead, the high-frequency telemetry stream was migrated from JSON to a compact binary packet format. JSON was kept only for low-frequency control and state messages, while sensor telemetry was packed into fixed-size binary structures. This reduced transmitted data volume, minimized parsing overhead, and improved the responsiveness of the real-time communication channel (prevented WebSocket queue growth, and lowered the processing load on the Android client).

- Real-Time Data Coordination

The system contains tasks with different timing and hardware requirements, including MPU processing, distance-sensor polling, telemetry preparation, POV rendering, CSV logging, and hammer actuation.

The firmware was reorganized into separate FreeRTOS tasks with explicit core assignments and priorities. High-priority tasks perform MPU processing and synchronized telemetry preparation, while lower-priority tasks handle distance polling, flash logging, and hammer actuation.

Shared sensor and system data are coordinated through protected writer updates and validated snapshots. Each accepted snapshot is used as the common source for telemetry packaging and CSV buffering, ensuring that the live and recorded outputs are derived from the same firmware state. Flash-memory access was moved out of the sampling path through RAM buffering, a dedicated staging buffer, and a lower-priority `loggerTask`. Block-based writes and deferred flushing further reduce the effect of LittleFS latency on time-sensitive execution.

A major conclusion from the integration stage was that real-time performance depends not only on fast sampling, but also on preventing blocking operations from interfering with critical timing loops. File writes, WebSocket transmission, JSON parsing, and UI updates can all introduce delays if they are executed directly inside time-sensitive paths.

The final software architecture reduces blocking by using separated tasks, controlled communication rates, compact binary telemetry, buffered logging, and synchronized data snapshots.

As a result, the system can acquire sensor data, update the POV display, stream telemetry, and store CSV data with improved timing stability and fewer interruptions to real-time behavior.

- Angle Tracking and POV Stability

The MPU-based integrated angle remained the primary phase reference for the final POV display because it provides the most reliable high-rate estimate during rotation.

VL-derived and fused angle estimates were retained as experimental analysis channels and recorded in the CSV file. Their ability to correct long-term drift was limited by lower sampling rates, fallback readings, and dependence on the surrounding geometry.

A timing-compensation term was introduced to reduce the delay between angle estimation and physical LED activation. This improved short-term POV alignment during changes in rotational speed, but it did not eliminate accumulated gyroscope drift.

- Android Graph Optimization

The Android graph initially suffered from performance issues caused by frequent data structure updates, repeated memory allocation, and excessive UI refreshes.

In real-time visualization, this caused visible stuttering, delayed graph updates, and temporary UI freezes.

The graphing mechanism was optimized by using a fixed-size sliding window and limiting UI updates to a stable refresh rate. Instead of recreating large data arrays or redrawing all samples after every incoming message, the app updates only the relevant visible data range.

This significantly reduced memory pressure, lowered garbage collection activity, and produced smoother real-time visualization.

- Dual-Network Operation

The ESP32 operates simultaneously in Access Point and Station modes, with each interface serving a different system function.

The Access Point interface provides a direct WebSocket connection for Android telemetry and control. The Station interface connects the ESP32 to the local network and allows a computer to retrieve the CSV file from LittleFS through the HTTP server.

This dual-network design supports both live mobile monitoring and independent file retrieval. It does introduce additional network-management complexity, but it is required by the current system workflow. Future development may simplify the architecture by transferring logged data through the existing WebSocket pipeline or another unified communication path.

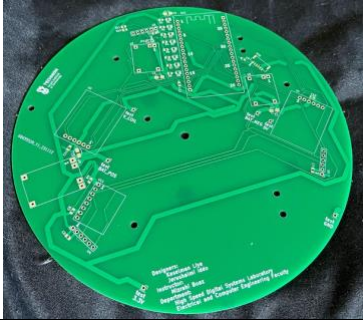
Overall, the optimized pipeline reduced blocking, communication overhead, memory pressure, and interference between tasks. As a result, sensor acquisition, POV rendering, Android telemetry, actuator control, and high-resolution CSV logging can operate concurrently with improved timing stability.

11.8 Component Reliability and Procurement Strategy

- Transition to More Reliable Components: Given the recurring failures of the charging and BMS modules, the primary conclusion is that more reliable and robust components must be considered for future iterations. The components used in this project proved too sensitive to operational loads and firmware flashing, highlighting the need for hardware that offers greater long-term stability.
- Inventory Availability and Redundancy: Experience has shown that one should not rely on a minimum quantity of critical components. It is highly recommended to order in bulk or utilize parts available in the lab's local stock. Having "backup components" on hand allows for the immediate replacement of modules or batteries damaged during development, preventing significant delays in the project timeline.

Integrated System

The following images present the fully integrated system, as well as an example of the live data output displayed on the computer. These visuals highlight the transition from design to implementation, demonstrating how the physical hardware closely follows the initial modeling and planning stages. The system includes all major components-sensors, motors, PCB, and structural elements-assembled and operational. The output screenshot illustrates successful communication between the hardware and software layers, confirming functionality such as sensor data transmission and live status feedback.

PCB	PCB – front side 		PCB – Back side 
Spinning Top	Spinning Top – Upper View 	Spinning Top – Bottom View 	Spinning Top – Side View 
Playing With the Design	Designing Cone 		Designing of Stabilization Device 
Final Prototype			

Output example:

- File Log (.CSV file):

The CSV log includes measurements and state values such as:

```
iter_global, activityType, raw_data_samp_time[ms], binLineWr_time[ms],  
iterTime_binlineIdx[ms]~20[ms]  
  
distLuna [cm], strengthLuna[Amp], tempLuna[Cdeg]  
  
mpuAngle[deg], mpuTotGyro[rad/sec], mpuTotAcc[m/s^2]  
mpu_gyroX[rad/sec] ~ -0.15[rad/sec], mpu_gyroY[rad/sec] ~ 0.72[rad/sec], mpu_GyroZ[rad/sec] ~  
30.42[rad/sec]  
mpu_accX[m/s^2] ~ 18.26[m/s^2], mpu_accY[m/s^2] ~ 36.41[m/s^2], mpu_accZ[m/s^2] ~ 10.27[m/s^2]  
  
vl1_dist[cm], vl2_dist[cm], vl3_dist[cm], VLAngle[deg]  
  
fusedAngle[deg], spincount, bucket10, digit  
currentTopDir, currentHammerDir, currentSystemMode, currentTopMode, currentHammerMode
```

Full output example is attached to [GitHub repository](#).

- Android Application



12. Thanks and Recognition

Mizrachi Boaz



and HSDSL lab's team: Orbach Mony, Rivkin Ina, Bulkin Lena

13. Bibliography

13.1. Sources to 3D files

[GrabCad Community](#)

13.2. Datasheets

attached to [GitHub repository](#)

13.3. ESP32 Safe Ports

Which ESP32 GPIOs are Safe to Use? [Lastminutesengineers](#)

14. Appendix

GitHub repository can be accessed by the following link:

[Spinning_Top_Real_Time_Platform](#)

Content of repository includes:

- Document Files
- Presentation Files
- Schematic Files
- Program Files
- Production Files
- ErrorSheets
- 3D Print Files
- 3D Modeling Files (Fusion360).
- KiCad Files
- DataSheets
- Components List
- BOM
- Web Files
- Media Files

Project Presentation can be accessed by the following link:

[Project Presentation Part 2](#)